

Java 并发面试题速记通关版 _ 面试刷题 mianshiya.com

本资源来自面试鸭: <https://www.mianshiya.com>

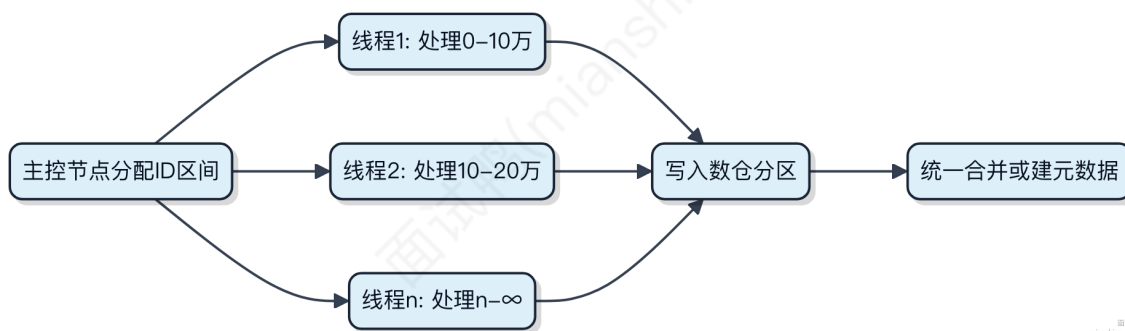
推荐更多免费学编程资源:

1. 编程导航学习网站: [学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库: [开始刷题, 面试遇原题!](#)
3. 精选简历模板大全: [1 分钟搞定简历!](#)
4. AI 资源导航网站: [获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试: [随时随地提升面试能力](#)

多线程并发同步数据时（数据库的数据同步到数仓中）需要注意什么问题？

多线程做数据库到数仓的数据同步，本质是在解决**一致性和性能**的平衡问题。你得先想清楚数据从哪来、怎么拉、怎么写。

- 1) 拉取数据时别直接 `SELECT * FROM table`，大表一查就卡住，还可能把数据库连接池搞崩。一般用自增 ID 或时间戳做分片，比如按 ID 范围切任务，每个线程处理一段，这样能并行拉数据。如果是 MySQL，可以考虑 `WHERE id BETWEEN ? AND ?` 配合索引。
- 2) 多个线程往数仓写数据，必须避免重复写或漏写。常见做法是每个线程处理的数据块带唯一标识，在数仓侧做幂等，比如 Hive 或 ClickHouse 写入前先检查分区是否存在，或者用事务型表引擎（如 Iceberg）提交时做原子替换。
- 3) 共享状态要小心。比如有个全局计数器记录同步进度，多个线程更新它就得加锁，但其实更好的方式是让每个线程独立汇报进度，由协调者汇总，压根不经过临界区。
- 4) 异常处理不能少。某个线程挂了，任务得能重试，但又要防止重复同步。通常把任务状态存到外部存储，比如 ZooKeeper 或数据库里的任务表，失败后可恢复。
- 5) 别忽视数据库的负载。并发太大，源库的 CPU、IOPS 可能扛不住。一般控制并发线程在 10~20 个以内，配合限流，像 Canal 或 DataX 实际也是这么干的。



Java 线程安全的集合有哪些？

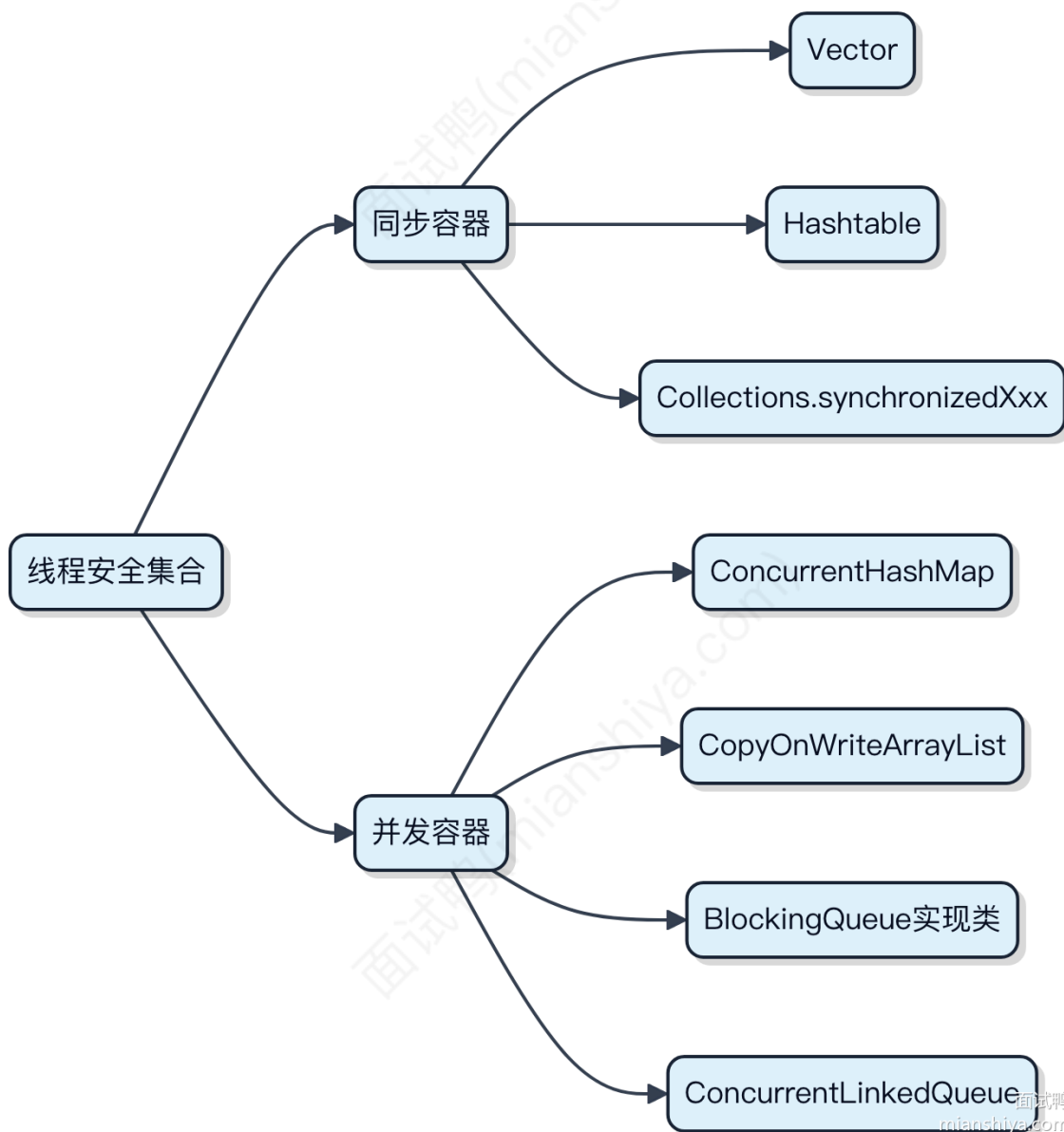
Java 里线程安全的集合主要分两类，一类是古董级别的同步容器，另一类是并发包里的高性能方案。

早期的 Vector 和 Hashtable 确实是线程安全的，方法都加了 synchronized，但问题是粒度太粗，同一时刻只能一个线程操作，性能差。Collections.synchronizedXxx 包装出来的集合也是类似机制，基本不推荐在新项目里用。

真正扛住高并发的是 java.util.concurrent (JUC) 包里的家伙。比如 **ConcurrentHashMap**，它把数据分成多个 segment (JDK 7) 或者用 CAS + synchronized 控制桶 (JDK 8+)，写操作基本不阻塞读，吞吐量能到 HashMap 的 80% 以上。实际开发中，只要涉及并发读写映射表，一律上 ConcurrentHashMap。

List 方面，CopyOnWriteArrayList 适合读多写少场景，比如监听器列表。每次修改都会复制整个数组，写代价大，但读完全无锁。Set 可以用 CopyOnWriteArraySet 或者用 ConcurrentHashMap 的 key 来模拟。

Queue 更丰富。BlockingQueue 接口下有一堆实现，ArrayBlockingQueue 是有界阻塞队列，基于数组；LinkedBlockingQueue 默认容量 2147483647，适合做生产者消费者缓冲；ConcurrentLinkedQueue 是无锁链表队列，非阻塞高吞吐。



Java 创建线程池有哪些方式？

Java 里创建线程池，最核心的方式就是通过 `ThreadPoolExecutor` 构造函数来搞，其他都是它的封装。

1) 直接用 `new ThreadPoolExecutor(...)`，把核心参数全配一遍：核心线程数、最大线程数、空闲存活时间、工作队列、拒绝策略。这种方式最灵活，Spring 的 `@Async` 底层就推荐自己定义这种线程池。

2) 用 `Executors` 工厂类，但它提供的几种快捷方式得小心：

- `newFixedThreadPool`：固定大小线程池，但用的是 `LinkedBlockingQueue`，队列无界，请求一多内存容易炸。
- `newCachedThreadPool`：看似弹性强，但最大线程数是 `Integer.MAX_VALUE`，瞬间高并发下会创建几万个线程，系统直接扛不住。
- `newSingleThreadExecutor`：本质就是单线程加无界队列，跟第一种一样有内存风险。
- `newScheduledThreadPool`：适合定时任务，类似 `Timer` 的升级版，可以用。

其实阿里巴巴规约里明确说了，**禁止使用 `Executors` 创建线程池**，就怕大家图省事踩坑。生产环境应该手动 `new ThreadPoolExecutor`，把每个参数都控制住。

```
new ThreadPoolExecutor(  
    4,           // 核心线程  
    8,           // 最大线程  
    60L,         // 空闲存活秒数  
    TimeUnit.SECONDS,  
    new ArrayBlockingQueue<>(100), // 有界队列  
    new ThreadPoolExecutor.CallerRunsPolicy() // 拒绝策略  
);
```

队列选 `ArrayBlockingQueue` 这种有界的，拒绝策略根据业务选，比如记录日志或让调用者自己执行。

Synchronized 修饰静态方法和修饰普通方法有什么区别？

`synchronized` 修饰普通方法时，锁的是当前实例对象，也就是 `this`。多个线程访问同一个对象的这个方法时才会互斥，如果是不同对象，压根不经过同一把锁，自然不互斥。

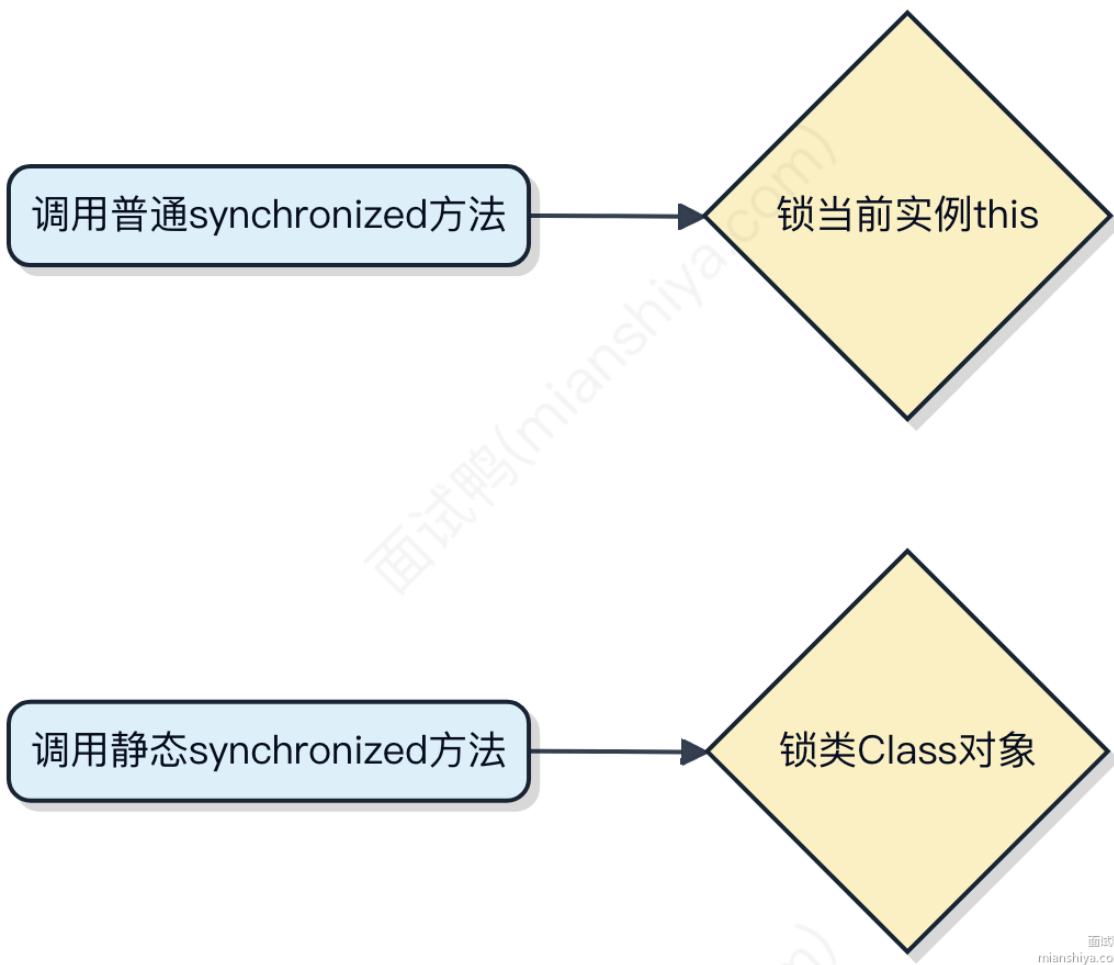
修饰静态方法时，锁的是类的 `Class` 对象，比如 `MyClass.class`。所有该类的实例调用这个静态方法时，都得竞争这一把锁。本质上是**类级别**的同步，和具体实例无关。

举个例子，你用 `synchronized` 修饰 `UserService` 的普通方法，那每个 `UserService` 实例都有自己的锁。但要是修饰的是静态方法，哪怕你 `new` 了 10 个 `UserService`，这 10 个实例也共用一把锁。

代码上看：

```
public class UserService {  
    public synchronized void instanceMethod() {  
        // 锁 this  
    }  
  
    public static synchronized void staticMethod() {  
        // 锁 UserService.class  
    }  
}
```

所以关键在于锁的粒度。普通方法锁实例，适合保护实例级别的状态；静态方法锁类，适合保护类级别的共享资源，比如静态缓存、计数器等。搞不定并发问题的时候，先想清楚你要保护的是实例数据还是全局数据。



Java 中 Thread.sleep(0) 的作用是什么？

让出 CPU 时间片，触发线程调度。

- 1) Thread.sleep(0) 并不会真正“睡”一段时间，而是向操作系统发出一个“我愿意放弃当前时间片”的信号。这时候调度器可以重新评估哪个线程该运行，避免当前线程一直霸占 CPU。
- 2) 在 JVM 层面，这会直接调用底层系统的纳秒级睡眠接口，传 0 表示不强制休眠时长，但依然会进入调度流程。有些系统上等价于 yield()，但更重量一点，因为它会进入阻塞状态再唤醒。
- 3) 典型使用场景是：你在写一个高频率轮询的循环，又不想完全吃满 CPU。比如早期 AQS 在抢锁失败时的自旋控制，或者某些无锁算法中主动降低竞争强度。

```
while (flag) {  
    if (condition()) break;  
    Thread.sleep(0); // 主动让出时间片，减少 CPU 消耗  
}
```

- 4) 和 Thread.yield() 的区别在于，yield() 只是建议调度器切换，而 sleep(0) 一定会进入定时器等待队列再被唤醒，开销稍大，但行为更确定。

这个操作在现代并发编程里很少手动写了，像 `LockSupport.park()` 或者 `synchronized` 优化后的自旋机制都做得更智能了。但在理解线程调度逻辑时，它是个很好的切入点。

Java 中如何创建多线程？

创建多线程在 Java 里最常用的就两种方式：继承 `Thread` 类和实现 `Runnable` 接口。其实本质上都是靠 `Thread` 对象来启动线程，只是任务定义的方式不同。

1) 直接 new Thread，重写 run 方法

这种方式简单直观，但一般不推荐，因为 Java 不支持多继承，你继承了 `Thread` 就没法再继承别的类了。

```
new Thread() {
    public void run() {
        System.out.println("我在子线程运行");
    }
}.start();
```

2) 实现 Runnable 接口，把任务传给 Thread

更灵活，解耦了任务和线程本身。你现在写的任务类可以交给 `Thread`，也可以交给线程池，比如提交给 `ThreadPoolExecutor`。

```
Runnable task = () -> System.out.println("通过 Runnable 执行");
new Thread(task).start();
```

3) 用 Callable + FutureTask 拿返回值

如果需要线程执行完后返回结果，就得用 `Callable`。它能抛异常、有返回值，配合 `FutureTask` 使用。

```
Callable<Integer> call = () -> {
    return 42;
};
FutureTask<Integer> future = new FutureTask<>(call);
new Thread(future).start();
Integer result = future.get(); // 阻塞等待结果
```

现在实际开发中，几乎不会手动 `new Thread`。高并发场景都用线程池，比如用 `Executors` 工厂类创建，背后其实是封装了 `ThreadPoolExecutor`，能复用线程、控制资源，避免频繁创建线程把系统搞崩。

为什么 Netty 不使用 ThreadLocal 而是自定义了一个 FastThreadLocal？

Netty 对性能的要求非常极致，特别是在高并发场景下处理海量连接时，连 `ThreadLocal` 的微小开销都可能成为瓶颈。JDK 原生的 `ThreadLocal` 在查找和写入时使用线性探测法，底层是哈希表结构，一旦发生哈希冲突就得不断探查，最坏情况时间复杂度接近 $O(n)$ 。更关键的是，每个 `ThreadLocal` 实例都要计算 hash code，还要处理扩容、内存泄漏等问题。

`FastThreadLocal` 换了个思路，它为每个 `FastThreadLocal` 分配一个唯一的索引下标，存储数据直接通过数组下标访问，相当于把哈希查找变成了 **数组寻址**，操作是 $O(1)$ 且稳定。这个数组挂在 `FastThreadLocalThread` 上，读写就像访问 `thread.array[index]` 一样快，压根不经过哈希计算和冲突处理。

这种设计在 Netty 这种需要频繁绑定上下文信息（比如事件循环中的任务上下文）的场景里优势明显。像在 `EventLoop` 中每个任务都需要隔离状态，用 `FastThreadLocal` 能让每次获取上下文的速度保持恒定，不会因为 key 多

了变慢。

代码上体现也很简单：

```
public class FastThreadLocal<T> {
    private final int index;

    public FastThreadLocal() {
        this.index = InternalThreadLocalMap.nextIndex();
    }

    public final void set(T value) {
        InternalThreadLocalMap threadMap = InternalThreadLocalMap.get();
        threadMap.setIndexedVariable(index, value);
    }
}
```

整个机制依赖 **预分配索引 + 数组直接存取**，牺牲了一点内存（每个线程持有较大数组），换来了极致的访问速度。对于 Netty 来说，这点内存代价完全值得。

说说 AQS 吧？

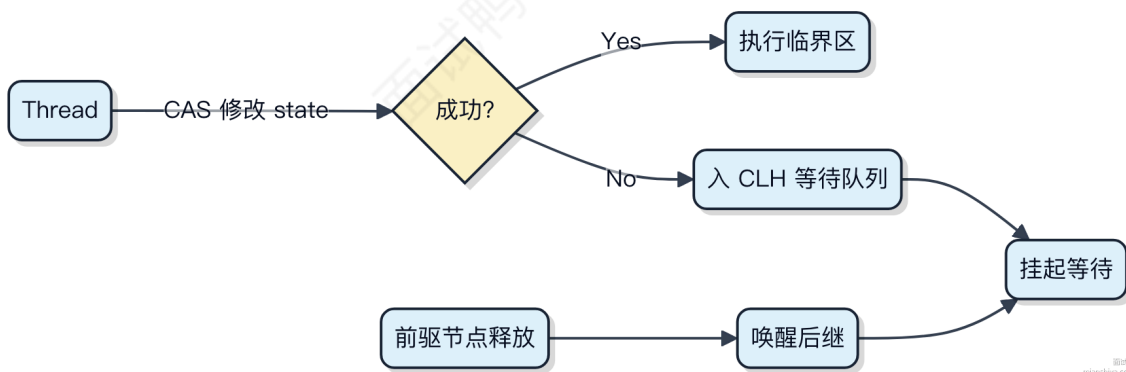
AQS，也就是 AbstractQueuedSynchronizer，是 Java 并发包里构建锁和同步器的底层基础。像 ReentrantLock、Semaphore、CountDownLatch 这些都是基于它实现的。

它的核心是用一个 volatile 修饰的 int 类型 state 来表示同步状态，比如加锁次数、信号量许可数。线程通过 CAS 去修改 state，改成功就拿到资源，改失败就进等待队列。

这个等待队列是个双向链表，也叫 CLH 队列，每个节点代表一个线程。没抢到锁的线程会被包装成 Node 入队，然后挂起。前面的线程释放资源后，会唤醒后继节点，实现公平调度。

AQS 把具体的获取和释放逻辑留给子类实现，比如 tryAcquire 和 tryRelease，自己只管排队和阻塞这些脏活累活。这种模板模式让开发者能方便地定制自己的同步组件。

- 1) 独占模式下，只有一个线程能拿到 state，比如 ReentrantLock
- 2) 共享模式下，多个线程可以同时拿到，比如 Semaphore 控制并发数
- 3) 支持公平和非公平策略，是否让新来的线程插队



注意 AQS 不是万能的，如果场景只是简单计数或一次性事件，直接用 JDK 提供的现成工具更稳妥，没必要重复造轮子。

ThreadLocal 的缺点？

ThreadLocal 看起来是个解决线程安全的银弹，但用不好反而会埋雷。最头疼的就是内存泄漏问题。每个线程的 ThreadLocalMap 里，键是弱引用指向 ThreadLocal 实例，值是强引用。一旦 ThreadLocal 实例被回收了，键变成 null，但值还占着内存，GC 拿它没辙。这种 entry 就成了“脏数据”，尤其在线程池场景下，线程长期存活，这些泄漏的对象越积越多，可能直接把堆撑爆。

- 1) 线程池复用线程时，如果不手动调用 remove，上一个任务的 ThreadLocal 数据可能被下一个任务误读，出现数据串扰。这在 Web 容器如 Tomcat 里特别常见，请求处理完不清理，下次可能拿到旧用户信息。
- 2) 大量使用 ThreadLocal 会增加单个线程的内存开销，尤其是存了大对象。本来想避免共享，结果每个线程都持有一份副本，内存占用翻 N 倍，N 就是活跃线程数，几百个线程就是几百份。
- 3) 调试困难。ThreadLocal 的数据隐式传递，堆栈里看不到上下文传递链，排查问题时得一层层去挖，日志也得手动注入上下文，否则 trace 不到源头。

```
// 正确姿势：务必 try-finally 中 remove
try {
    userIdHolder.set(123L);
    // 业务逻辑
} finally {
    userIdHolder.remove(); // 清理，避免泄漏和污染
}
```

所以 ThreadLocal 适合传生命周期短、必清理的上下文，比如一次请求的用户身份、事务 ID。像 TransmittableThreadLocal 这种能解决线程池传递的工具，底层也是靠包装任务做自动清理，本质还是补救措施。

Java 中的 InheritableThreadLocal 是什么？

InheritableThreadLocal 解决的是父子线程间 ThreadLocal 数据传递的问题。普通 ThreadLocal 在子线程里是拿不到值的，因为每个线程的 threadLocals 是独立的 map。

但 InheritableThreadLocal 能让子线程创建时，把父线程当前的 inheritableThreadLocals 里的数据拷贝一份到自己身上。这个动作发生在 new Thread() 的时候，调用的是 Thread 类里的 init 方法。

有两个关键点要注意：1) 它是一次性拷贝，父线程后续修改不影响已创建的子线程。2) 子线程再创建子子线程也能继续传递，但中间如果父线程变了，下层线程也不会感知。

常见于需要上下文透传的场景，比如日志链路追踪 ID、用户登录上下文。像阿里开源的 TLog 或 SkyWalking 的上下文传播，底层就会用到它做基础支撑。

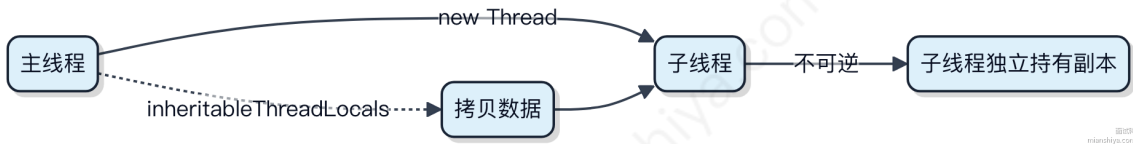
不过在池化线程场景下会出问题，因为线程复用导致可能继承到上一个任务的脏数据。这时候就得靠 Alibaba 的 TransmittableThreadLocal 来补救，它通过重写线程池的提交逻辑来实现动态传递。

代码上看，其实就是重写了 childValue 方法：

```
public class InheritableThreadLocal<T> extends ThreadLocal<T> {
    protected T childValue(T parentValue) {
        return parentValue;
    }
}
```

```
}  
}
```

整个机制依赖 Thread 类内部两个字段：threadLocals 和 inheritableThreadLocals，都是由 ThreadLocalMap 持有。



Java 中使用 ThreadLocal 的最佳实践是什么？

ThreadLocal 本质是给每个线程提供独立的变量副本，避免共享带来的并发问题。用好了是神器，用不好就是内存泄漏的源头。

使用时第一个要注意的是**初始化**。别直接 new ThreadLocal() 就用，尽量重写 initialValue() 方法，让第一次 get 时自动填充默认值。比如做日期格式化工具时，可以这样：

```
private static final ThreadLocal<SimpleDateFormat> formatter =  
    new ThreadLocal<SimpleDateFormat>() {  
        @Override  
        protected SimpleDateFormat initialValue() {  
            return new SimpleDateFormat("yyyy-MM-dd");  
        }  
    };
```

重点在于清理。线程池里的线程会复用，如果不手动 remove，那这个 ThreadLocalMap 里的 Entry 就一直占着内存。尤其在 Web 服务这种基于线程池的场景下，请求一多，内存早晚被撑爆。所以通用做法是在 finally 块里执行 remove。

- 1) 用完立刻清理，尤其是在线程池环境中
- 2) 优先使用静态 final 修饰 ThreadLocal 实例，防止到处乱声明
- 3) 能用局部变量或入参传递解决的，就别上 ThreadLocal

还有个坑是父子线程数据传递。原生 ThreadLocal 不支持，这时候得换 InheritableThreadLocal，但它对线程池也不友好。真要跨线程传上下文，推荐阿里开源的 TransmittableThreadLocal，专门修了这问题。

最后提醒一点，别拿 ThreadLocal 当缓存用。它不是为高性能存储设计的，存大对象或者海量 key 肯定搞不定。

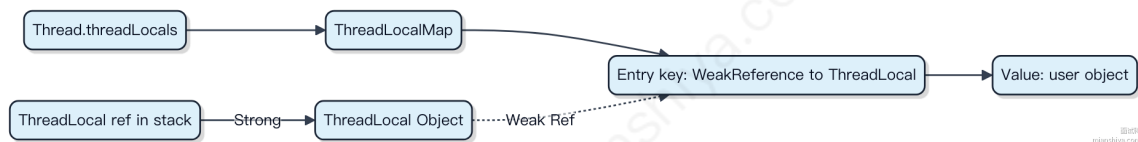
为什么 Java 中的 ThreadLocal 对 key 的引用为弱引用？

ThreadLocal 的 key 是弱引用，主要是为了解决内存泄漏的隐患。每个线程都有自己的 ThreadLocalMap，这个 map 的 key 就是 ThreadLocal 实例，value 是线程本地的值。如果这里用强引用，那么即使 ThreadLocal 实例在外部已经被置为 null，只要线程还没结束，map 里的 key 依然能被访问到，GC 就没法回收它，造成内存泄漏。

使用弱引用后，当外部没有强引用指向 ThreadLocal 时，下一次 GC 就能把 key 回收掉，这时候 key 变成 null，但 value 还在 map 里，这就是所谓的“脏 entry”。如果不处理，value 依然会累积。所以真正靠得住的做法是在调用 remove() 主动清理，像 Tomcat 复用线程池时就特别注意这点，不然可能几十个请求就搞不定内存。

虽然 key 是弱引用，但不能完全依赖它来避免内存泄漏。最佳实践是每次用完 ThreadLocal 都显式调用 remove()，尤其是在高并发、长生命周期的线程场景下，比如使用线程池时。

- 1) 弱引用让 key 可被回收，防止 ThreadLocal 实例本身泄漏
- 2) 但 value 不会被自动清理，必须手动 remove()
- 3) 不及时清理会导致 Entry 对象堆积，最终 OOM



不要以为用了弱引用就万事大吉，**弱引用**只是减轻了泄漏风险，真正的安全还得靠主动清理。

Java 中的 ThreadLocal 是如何实现线程资源隔离的？

每个线程对 ThreadLocal 的访问都通过一个专属的 **ThreadLocalMap** 结构来存储数据，这就像是给每个线程配了一个独立的储物柜。

- 1) 每个 Thread 实例内部持有一个 ThreadLocalMap，key 是 ThreadLocal 实例的弱引用，value 是线程本地的值。
- 2) 调用 `threadLocal.set(value)` 时，JVM 会找到当前线程的 map，以当前 ThreadLocal 为 key 存入 value。
- 3) 调用 `threadLocal.get()` 时，也是从当前线程的 map 里取出对应 key 的值，自然拿不到其他线程的数据。

```
private ThreadLocal<String> userId = new ThreadLocal<>();

// 线程 A
userId.set("user001");
System.out.println(userId.get()); // user001

// 线程 B
userId.set("user002");
System.out.println(userId.get()); // user002, 不受 A 影响
```

这里的关键是 **内存结构隔离**，不是靠锁或者同步机制。每个线程操作的是自己栈和堆中独立的部分，压根不经过共享区域。

常见坑是内存泄漏：虽然 key 是弱引用，但 value 是强引用。如果线程长期运行且 ThreadLocal 被置空，map 里的 entry 可能 still hold 着 value，导致回收不了。所以用完记得调用 `remove()`。

适用场景比如 Web 项目中用 ThreadLocal 存储用户登录信息（像 Spring Security 就这么干），或者数据库事务管理中绑定当前连接。但别滥用，尤其在线程池环境下，必须手动清理。

为什么在 Java 中需要使用 ThreadLocal？

ThreadLocal 的本质是给每个线程提供一个独立的变量副本，避免多线程下对共享变量的竞争。你不用加锁，也能实现线程安全。

- 1) 每个线程访问自己的数据，彼此隔离。比如在 Web 应用里，一个请求对应一个线程，你想在整个调用链中传递用户信息或上下文，又不想一层层手动传参，就可以把用户信息存到 ThreadLocal 里。

2) 典型场景是 Spring 的事务管理。同一个事务要求在同一线程内使用同一个数据库连接，Connection 就通过 ThreadLocal 绑定到当前线程，后续从数据源获取时直接命中，压根不经过重新创建。

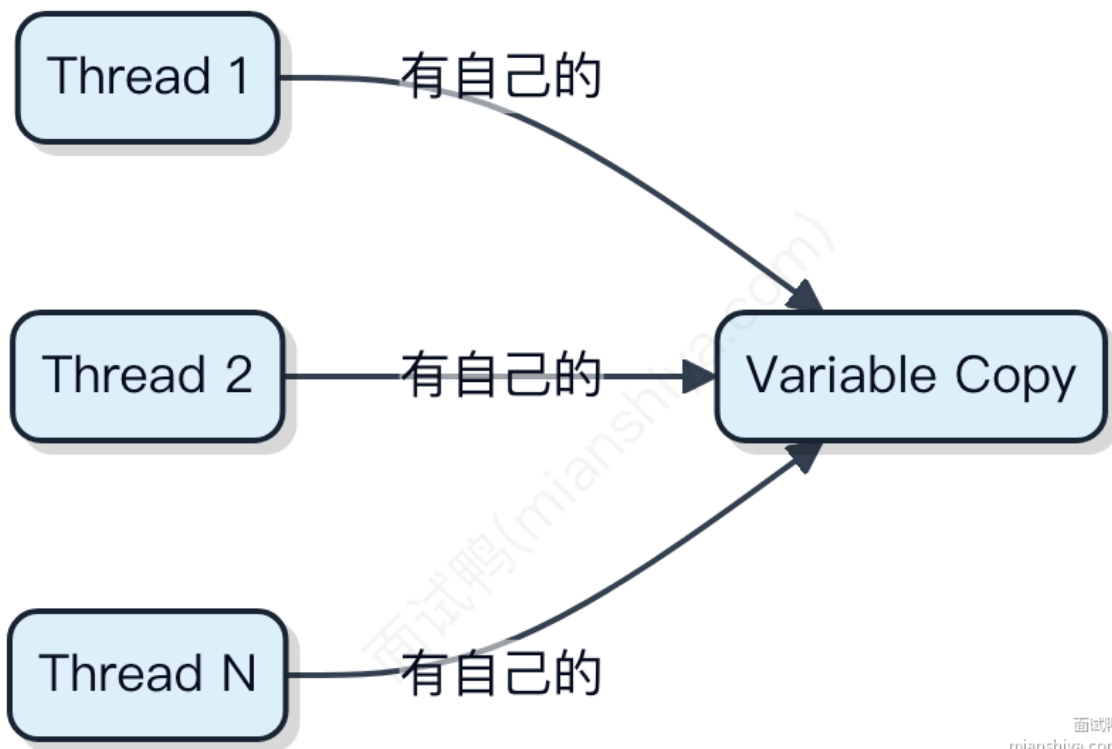
3) 内存泄漏是个大坑。ThreadLocal 底层用了一个弱引用的 Entry 数组，但 key 是弱引用，value 不是。如果 ThreadLocal 实例被回收了，key 变成 null，但 value 还占着内存，这时候就发生泄漏。所以用完一定要调 remove()。

代码上很简单：

```
private static final ThreadLocal<UserInfo> context = new ThreadLocal<>();

// 设置
context.set(userInfo);
// 获取
UserInfo user = context.get();
// 清理
context.remove();
```

Tomcat 线程池复用线程时，如果不清理，可能拿到的是上个请求的残留数据，出问题很难排查。所以 set 之后必须 finally 块里 remove。



Java 中的 final 关键字是否能保证变量的可见性？

final 关键字在 Java 中不仅能防止变量被重新赋值，还能在多线程环境下**保证可见性**，但这依赖于正确的使用场景。

1) 对于引用类型，final 能确保对象初始化完成后，其他线程看到的都是完全构造好的实例。比如单例模式中使用 `private final` 字段，配合构造完成后的发布，不需要额外同步就能安全共享。

2) Java 内存模型规定, final 字段的写操作不会被重排序到构造方法之外。这意味着一旦对象引用对其他线程可见, 其所有 final 字段的值也一定可见。

3) 非 final 字段则没有这个保障。即使主线程构造完对象并发布引用, 其他线程仍可能读到未初始化或部分初始化的字段值。

代码示例:

```
public class FinalFieldExample {
    final int x;
    int y;

    public FinalFieldExample(int x, int y) {
        this.x = x; // final 写
        this.y = y; // 普通写
    }
}
```

只要对象是正常构造 (没有 this 逸出), x 的值对所有线程都可见且一致。

但注意, final 不适用于后续修改。如果想让非 final 字段也有可见性, 得靠 volatile 或 synchronized。



Java 中的线程安全是什么意思?

线程安全指的是多个线程同时访问某个类或方法时, 其行为仍然正确, 不会因为线程调度的交错而导致数据错误或状态不一致。关键在于**共享变量**和**竞态条件**。

- 1) 如果一个类没有共享状态, 比如全是局部变量, 那它天生就是线程安全的。
- 2) 如果有共享状态, 比如成员变量或静态变量, 就需要通过同步手段来保护, 否则多个线程读写同一块内存就可能出问题。典型的例子是两个线程同时对 `int counter` 执行自增, 结果可能只加了一次。

常见的保障方式包括:

- 使用 `synchronized` 关键字控制临界区
- 用 `java.util.concurrent` 包下的原子类, 比如 `AtomicInteger`
- 采用 `ThreadLocal` 隔离变量副本
- 使用并发容器如 `ConcurrentHashMap` 替代 `HashMap`

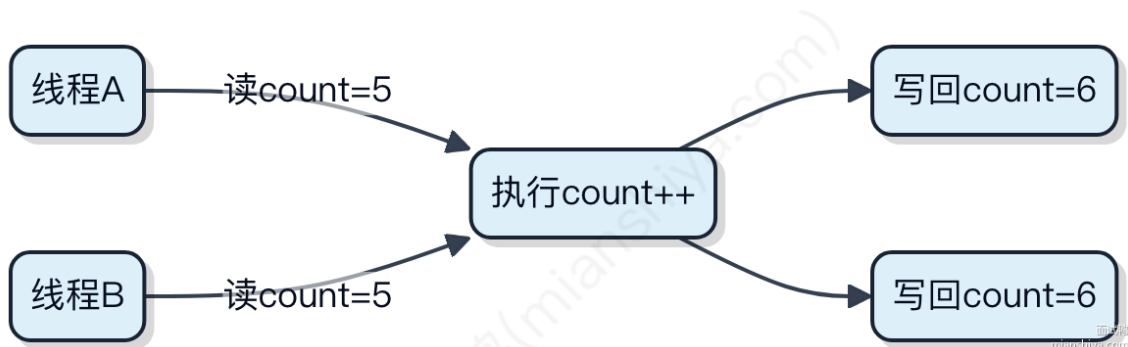
```
public class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++; // 操作需要原子性
    }
}
```

上面这个例子中, `synchronized` 确保了 `increment` 方法在同一时刻只能被一个线程执行, 压根不经过锁的线程进不来, 从而避免了脏写。

反过来，像 ArrayList 就不是线程安全的，多线程下扩容可能引发死循环，这时候就得换成 CopyOnWriteArrayList 或者加锁处理。搞不定并发问题的代码，在高并发场景下很容易崩。

典型场景对比



你了解 Java 中的读写锁吗？

读写锁的核心在于允许多个读操作并发进行，但写操作必须独占。这种设计在读多写少的场景下能显著提升性能，比如缓存系统或配置中心。

Java 里的 `ReentrantReadWriteLock` 就是典型实现。它内部维护了一对锁，一个读锁和一个写锁。读锁是共享的，写锁是排他的。多个线程可以同时持有读锁，只要没有线程在写。

- 1) 当有线程持有写锁时，其他任何线程都无法获取读锁或写锁
- 2) 当有线程持有读锁时，其他线程可以继续获取读锁，但不能获取写锁
- 3) 写锁可以降级为读锁，但读锁不能升级为写锁，否则会死锁

代码上很简单：

```
ReentrantReadWriteLock lock = new ReentrantReadWriteLock();
Lock readLock = lock.readLock();
Lock writeLock = lock.writeLock();

// 读操作
readLock.lock();
try {
    // 读取共享资源
} finally {
    readLock.unlock();
}

// 写操作
writeLock.lock();
try {
    // 修改共享资源
} finally {
    writeLock.unlock();
}
```

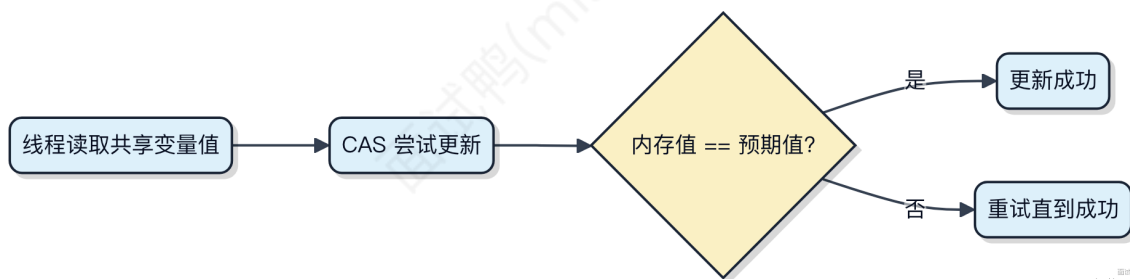
要注意的是，**锁降级**需要手动控制顺序：先获取写锁，再获取读锁，然后释放写锁。反过来就不行。

使用不当容易造成饥饿问题，比如写线程一直抢不到锁，因为读线程太多。这时候可以考虑 `StampedLock`，它支持乐观读，性能更好，但使用复杂度也更高。

什么是 Java 的 CAS (Compare-And-Swap) 操作?

CAS (Compare-And-Swap) 是 Java 并发编程里实现原子操作的一种底层机制，它直接由 CPU 提供指令支持，比如 x86 的 `cmpxchg` 指令。这个操作包含三个参数：内存位置 V、预期旧值 A 和要更新的新值 B。只有当 V 位置的当前值等于 A 时，才会把 V 更新为 B，否则什么都不做。整个过程是原子的，不会被线程调度打断。

Java 里的 `Unsafe` 类封装了这些底层操作，而我们日常用的 `AtomicInteger`、`AtomicLong` 等原子类就是基于它实现的。比如调用 `incrementAndGet()`，底层其实就是不断用 CAS 尝试加 1，直到成功为止。



这种机制避免了传统锁的阻塞问题，适合**多读少写、竞争不激烈**的场景。但也有几个坑得注意。1) ABA 问题，就是值从 A 变成 B 又变回 A，CAS 会误判没变过，可以用 `AtomicStampedReference` 带版本号解决。2) 自旋开销，如果冲突频繁，线程会一直重试，CPU 占用可能飙高。3) 只能保证单个变量的原子性，多个变量就得上锁或者用 `synchronized`。

代码上很简单：

```
private AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet(); // 底层就是 CAS 自旋
```

所以你看，像 `ConcurrentHashMap`、`AQS` 这些高并发组件，底层都大量依赖 CAS 来扛住并发压力，干的都是无锁同步的脏活累活。

什么是 Java 的 TransmittableThreadLocal?

`ThreadLocal` 在父子线程间传递数据时会断掉，比如主线程 set 一个值，提交到线程池的子线程里就拿不到了。这在像 `TraceId` 透传、用户上下文传递这种全链路追踪场景下直接就搞不定了。

TransmittableThreadLocal 就是为了解决这个问题生的。它能在任务提交到线程池时自动把当前线程的 `ThreadLocal` 值拷贝到子线程，等任务执行完再恢复，整个过程对业务代码透明。

常见于像 `Hystrix`、`Dubbo`、`SkyWalking` 这些框架里做上下文透传。比如你用 Alibaba 的 TTL 库，只需要把原来的 `ThreadLocal` 换成 `TtlCallable` 或 `TtlRunnable` 包一下提交的任务就行。

```
TtlRunnable ttlRunnable = TtlRunnable.get(() -> {
    System.out.println("child: " + TransmittableThreadLocalHolder.userId.get());
});
```

- 1) 核心原理是在 submit 任务时 capture 当前线程的所有 TransmittableThreadLocal 值
- 2) 执行前 re-play 到 worker 线程，执行完 clear 掉避免内存泄漏
- 3) 底层靠的是重写线程池的 execute/submit 方法，或者用 Java Agent 字节码增强



注意别滥用，频繁提交短期任务时 capture/copy 开销不小。优先用于必须透传的场景，比如日志链路追踪、权限上下文。

Java 中的 wait、notify 和 notifyAll 方法有什么作用？

wait、notify 和 notifyAll 是 Java 中用于线程间协作的机制，它们定义在 Object 类里，不是 Thread 的方法，这点很多人一开始会搞混。

这三个方法必须在 synchronized 块或方法中调用，因为它们依赖于对象的**内置锁 (monitor lock)**。一个线程只有持有该锁，才能安全地进入等待状态或唤醒其他等待者。

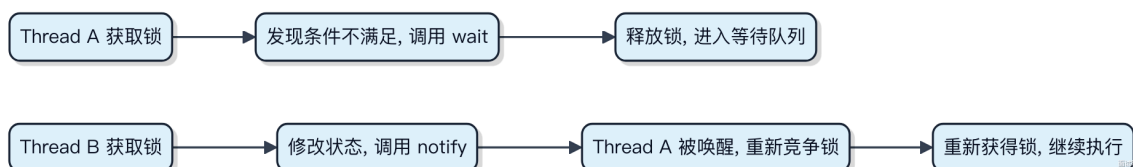
- 1) wait() 会让当前线程释放锁并进入阻塞状态，直到被其他线程通过 notify 或 notifyAll 唤醒。它相当于说“我暂时干不了活了，先歇着，有事叫我”。
- 2) notify() 会从该对象的等待队列中随机唤醒一个线程。注意是“一个”，不保证顺序，而且被唤醒的线程要重新竞争锁才能继续执行。
- 3) notifyAll() 则唤醒所有在该对象上等待的线程。虽然看起来浪费，但在生产者-消费者这类场景里更安全，比如 BlockingQueue 的实现通常就用 notifyAll 避免线程饥饿。

典型使用模式像这样：

```
synchronized (lock) {  
    while (conditionNotMet) {  
        lock.wait();  
    }  
    // 处理逻辑  
}
```

这里用 while 而不是 if，是因为可能存在**虚假唤醒 (spurious wakeup)**，即使没人通知，线程也可能自己醒来。

如果多个线程等待不同条件，用 notify 可能只唤醒了一个无关线程，导致死锁。这种情况下必须用 notifyAll，或者改用 ReentrantLock 配合 Condition，控制更精细。



Java 中什么情况会导致死锁？如何避免？

死锁最经典的场景就是两个线程各自持有对方需要的锁，然后互相等。比如线程 A 拿了锁 1，想去拿锁 2；线程 B 拿了锁 2，却在等锁 1。结果谁也别想过去，卡死了。

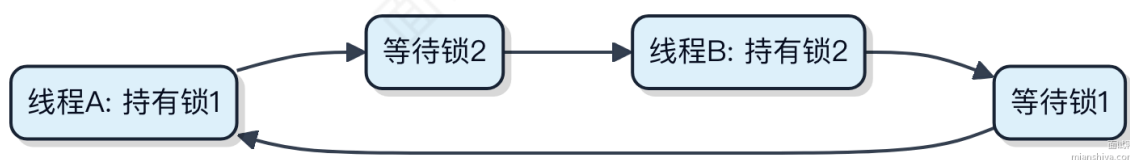
这种情况要发生，得同时满足四个条件：

- 1) 互斥：锁这东西本来就是排他的，同一时间只能一个线程用
- 2) 占有并等待：我已经拿着一个锁了，还等着另一个
- 3) 不可抢占：别人不能强行把我的锁拿走
- 4) 循环等待：A 等 B，B 等 C，C 又等 A，形成闭环

只要打破其中一个，死锁就搞不定你。

最常见的避免方式是**统一线程加锁顺序**。比如大家都约定先申请 ID 小的锁，再申请 ID 大的。这样就不会出现 A→B 而 B→A 的反向依赖。

还有一个办法是使用 tryLock 带超时，不傻等。比如用 ReentrantLock，最多等 5 秒，拿不到就释放已有资源，退回来重试或者报错，而不是死磕。



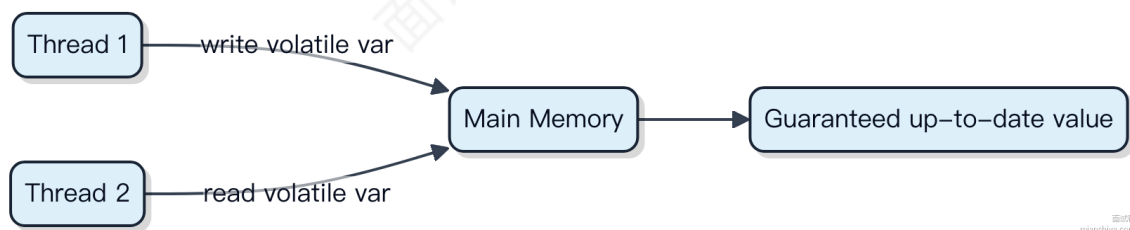
实际开发里，像数据库的行锁、Redis 分布式锁处理不当都容易出这问题。Spring 里事务方法嵌套调用，如果加锁顺序乱来，也可能中招。最稳妥的还是设计阶段就想清楚资源依赖关系，别让锁的获取路径打结。

Java 中 volatile 关键字的作用是什么？

volatile 关键字主要解决多线程环境下的可见性和指令重排序问题。当一个变量被声明为 volatile，JVM 会保证这个变量的修改对所有线程立即可见，不会因为线程本地缓存导致读取到过期值。

- 1) 每次读取 volatile 变量都会直接从主内存加载，每次写入也会立刻刷新回主内存，绕开了线程的工作内存。这使得一个线程的修改能被其他线程及时感知。
- 2) 编译器和处理器不会对 volatile 写操作与其前面的读写操作进行重排序，同样也不会对 volatile 读操作与其后面的读写操作重排。这种内存屏障机制保证了代码执行顺序与程序逻辑一致。

但要注意，volatile 不保证原子性。比如 i++ 这种复合操作，即使变量是 volatile 的，依然可能出错。需要原子性时得用 synchronized 或 AtomicInteger 这类工具。



典型使用场景是标志位控制，比如用 volatile boolean running = true; 来控制循环是否继续。像 Double-Checked Locking 模式中，单例对象引用也必须用 volatile 修饰，防止对象还没初始化完就被其他线程读到。

别拿它当 synchronized 用，搞不定复杂同步逻辑。简单状态标记、一次性安全发布，这些才是它的主战场。

什么是 Java 中的线程同步？

多个线程同时访问共享资源时，如果不加控制，数据就可能被改乱。比如两个线程同时对一个变量做 `i++`，结果可能只加了一次，这就是**竞态条件**。

要解决这个问题，就得让线程排队访问，这就是线程同步。Java 最常用手段是 **synchronized** 关键字，它能保证同一时刻只有一个线程能执行某段代码。

比如用 `synchronized` 修饰方法或代码块：

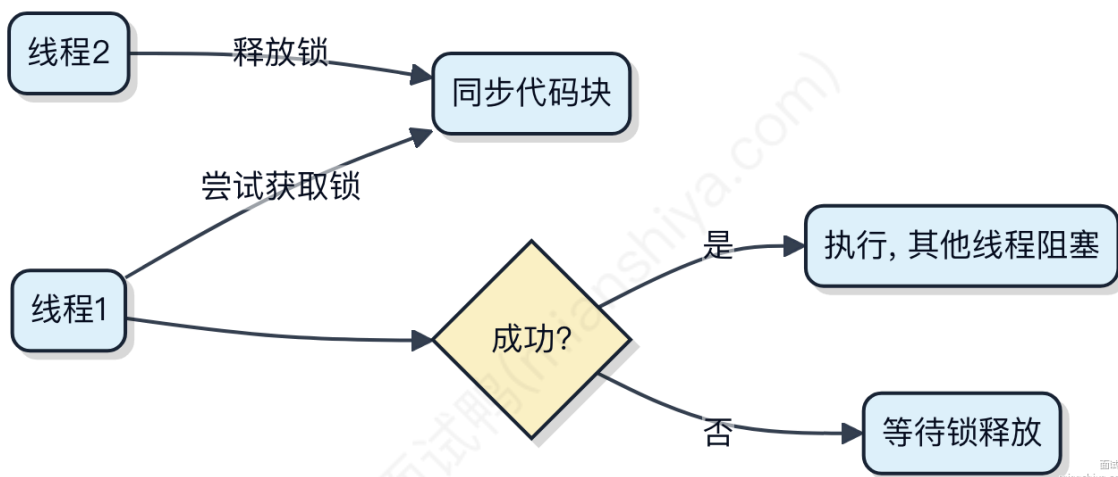
```
synchronized (this) {  
    count++;  
}
```

JVM 会通过对象的**监视器锁 (Monitor)** 来实现，线程进入时尝试获取锁，拿不到就阻塞，拿到才能往下走，执行完自动释放。

除了 `synchronized`，还可以用 `ReentrantLock` 这种显式锁，灵活性更高，比如支持超时、可中断，但代码也更复杂。像 `ConcurrentHashMap` 就是用 `CAS + synchronized` 的组合来兼顾性能和安全。

- 1) `synchronized` 是 JVM 层面支持的，编译后会生成 `monitorenter` 和 `monitorexit` 指令
- 2) 锁升级过程：无锁 → 偏向锁 → 轻量级锁 → 重量级锁，减少竞争不激烈时的开销
- 3) 锁的是对象，不是代码，多个线程抢同一个对象的锁才会互斥

别忘了 `volatile`，它不能保证原子性，但能禁止指令重排，适合状态标记场景。



Synchronized 能不能禁止指令重排序？

`synchronized` 主要解决的是互斥性和可见性问题，它能保证同一时刻只有一个线程进入同步块，同时对共享变量的修改对后续进入的线程可见。至于指令重排序，它确实能限制一部分，但不是靠插入内存屏障那种精细控制。

Java 内存模型里，**synchronized** 的实现依赖于 `monitor enter` 和 `exit`，`monitor enter` 会触发一个 `acquire` 操作，`exit` 会触发 `release` 操作。`acquire-release` 语义能保证：在 `monitor exit` 之前的所有写操作，对下一次 `acquire` 同一把锁的线程来说都是可见的，并且不会被重排序到 `exit` 之后。同理，`acquire` 之后的操作也不会被重排序到它之前。

这相当于在锁释放时，把之前的写操作都“刷出去”，在获取锁时，强制重新加载共享变量。所以从效果上看，多个线程通过同一把锁同步时，不会看到彼此内部的指令重排结果。

但它不像 volatile 那样针对单个变量做读写屏障，它的边界是整个同步块。你不能指望 synchronized 去禁止某个特定变量的重排序，如果需要更细粒度控制，得用 volatile 或显式内存屏障。

- 1) 锁的作用范围是代码块，不是变量
- 2) 它靠 acquire-release 语义间接抑制重排
- 3) 比 volatile 更重，但适合复合操作



说白了，它能“顺便”管住重排序，但不是为此设计的。

Java 线程池内部任务出异常后，如何知道是哪个线程出了异常？

线程池里的任务抛了异常，如果没做特殊处理，**异常信息会直接打印到控制台**，但不会让线程池本身崩溃。默认情况下，ThreadPoolExecutor 会捕获任务执行中的 Throwable，然后调用 worker 的 afterExecute 方法。

想知道是哪个线程出了问题，最直接的办法就是重写 afterExecute 这个钩子方法。这个方法在任务执行完成后被调用，无论成功还是抛异常。你可以在这里拿到 Runnable 和可能抛出的 Throwable。

- 1) 可以在线程池提交任务前，把任务包装一下，在里面加 try-catch，手动记录线程名和堆栈。
- 2) 更通用的做法是继承 ThreadPoolExecutor，覆盖 afterExecute 方法，在里面打印当前线程名称和异常堆栈。

```
@Override
protected void afterExecute(Runnable r, Throwable t) {
    if (t != null) {
        System.err.println("线程 [" + Thread.currentThread().getName() + "] 执行任务时出错: " + t);
        t.printStackTrace();
    }
}
```

3) 使用 submit 提交任务返回 Future，通过 future.get() 拿到 ExecutionException，再 getCause() 获取原始异常。这种方式适合需要精确控制异常处理的场景。

其实最常见的做法还是结合日志 MDC，在任务开始时记录线程名、任务ID等上下文，这样即使异常被捕获，日志里也能对上号。像 Dubbo、RocketMQ 内部都是这么干的，出了问题一查日志就知道谁搞的。

Java 线程池中 shutdown 与 shutdownNow 的区别是什么？

调用 shutdown 后，线程池状态变为 SHUTDOWN，不再接受新任务，但会继续处理队列里的任务。正在执行的线程不会被中断，等它们跑完，队列空了，线程池才真正终止。

shutdownNow 则更激进，状态变成 STOP，同样不接收新任务，但它会尝试**中断**所有工作线程，包括正在运行的，并且把队列里还没处理的任務全部取出，返回给调用者。

所以关键区别在于是否中断进行中的任务，以及对任务队列的处理方式。

1) shutdown 是优雅关闭，任务都能跑完，适合应用正常停机时使用，比如 Spring 容器关闭时触发。2) shutdownNow 是强制关闭，可能有任务直接被打断，适用于需要快速响应终止的场景，比如服务健康检查失败要立即退出。3) 两者都不等线程池真正结束就返回，如果想同步等待，得配合 awaitTermination。

代码上长这样：

```
executor.shutdown();  
// 或  
List<Runnable> remaining = executor.shutdownNow();
```

如果想确保清理彻底，一般先 shutdown，再调用 awaitTermination 设定超时，超时后仍没结束再考虑 shutdownNow 强制干预。



Java 中 Thread.sleep 和 Thread.yield 的区别？

sleep 和 yield 都是线程让出执行权的方法，但行为完全不同。

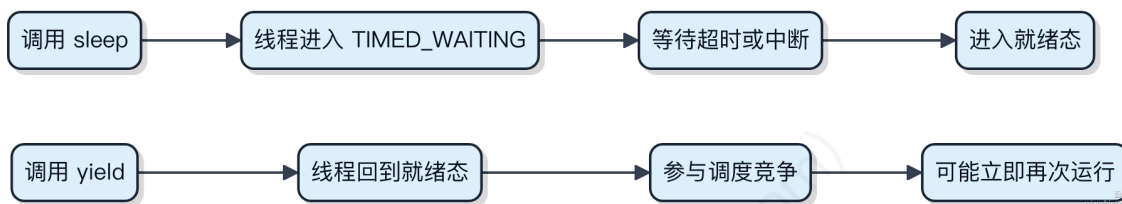
1) Thread.sleep(long millis) 让当前线程**强制进入阻塞状态**，指定时间内不会参与 CPU 竞争。时间一到，线程进入就绪态，等待调度。这期间其他线程，包括优先级更低的，都能获得执行机会。比如你写个定时任务轮询，可以用 sleep 控制间隔。

```
try {  
    Thread.sleep(1000); // 休眠 1 秒  
} catch (InterruptedException e) {  
    Thread.currentThread().interrupt();  
}
```

2) Thread.yield() 只是建议当前线程**让出 CPU**，从运行态回到就绪态，是否生效完全取决于操作系统调度器。它不保证任何时间的暂停，甚至可能被立刻再次选中。一般很少用，因为效果不可控。JVM 实现上，yield 通常会触发一次线程调度尝试，但不是阻塞。

关键区别在于：**sleep 会让线程阻塞一段时间，yield 只是打个招呼说“我愿意让”**。sleep 会释放 CPU 但不释放锁，yield 同样不释放锁，但压根不经过阻塞队列。

实际开发中，sleep 常用于限流、重试间隔等场景，而 yield 基本见不到，除非在极特殊的性能调优场景下尝试减少线程竞争开销，但效果通常不如直接用并发工具类。



在 Java 中主线程如何知晓创建的子线程是否执行成功？

主线程要感知子线程的执行结果，本质是做**线程间通信**，得靠共享状态或回调机制。

1) 最直接的方式是用 `Future + Callable`。子线程执行任务后返回结果，主线程通过 `get()` 拿结果或捕获异常。

```
Future<String> future = executor.submit(() -> {
    // 子线程逻辑
    return "success";
});
String result = future.get(); // 阻塞等待结果
```

2) 如果用 `Thread` 原生方式，可以通过共享变量加 `volatile` 保证可见性，子线程执行完修改状态，主线程轮询或等待通知。

```
volatile boolean success = false;
new Thread(() -> {
    try {
        // 执行任务
        success = true;
    } catch (Exception e) {
        success = false;
    }
}).start();
```

3) 更优雅的是用 `CountDownLatch`，让主线程阻塞等待子线程完成。适用于只关心“是否完成”的场景。

```
CountDownLatch latch = new CountDownLatch(1);
new Thread(() -> {
    try {
        // 业务逻辑
        latch.countDown();
    } catch (Exception e) {
        latch.countDown();
    }
}).start();
latch.await(); // 主线程等待
```

4) 实际开发中，像 `Spring` 的 `@Async` 或 `CompletableFuture` 会更常用。后者能链式处理结果，支持成功和异常回调。

```
CompletableFuture.supplyAsync(() -> {
    return "result";
}).whenComplete((res, ex) -> {
    if (ex == null) {
        System.out.println("成功: " + res);
    } else {
        System.out.println("失败: " + ex.getMessage());
    }
});
```

关键在于，不要靠 `isAlive()` 或 `join()` 判断成功与否，它们只能知道线程状态，不知道业务成败。真正可靠的是拿到返回值或异常信息。

volatile 与 Synchronized 的区别是什么？

volatile 和 synchronized 都能保证多线程下的可见性，但它们的职责和实现机制差得远了。

volatile 关键字只保证变量的**可见性**和**禁止指令重排**，不保证原子性。只要一个线程修改了 volatile 变量，新值会立刻刷到主内存，其他线程读这个变量时也直接从主内存拿。底层靠的是**内存屏障**（Memory Barrier）来实现，比如在写 volatile 变量前插入 StoreStore 屏障，后面加 StoreLoad 屏障。

synchronized 则是重量级的同步机制，它保证可见性、原子性和有序性。它通过**对象监视器**（monitor）实现，进入同步块前必须获取锁，执行完后释放。同一时刻只有一个线程能进入临界区。它的可见性是因为解锁前会把变量刷新回主内存。

- 1) volatile 适合状态标志位场景，比如 `private volatile boolean running = true;`，多个线程根据这个值决定是否退出循环。
- 2) synchronized 更适合复合操作，比如 `i++`，即便 `i` 是 volatile 的，自增也不是原子的，还是得上锁。
- 3) 性能上，volatile 几乎没有锁开销，而 synchronized 在 JDK 6 之后做了优化，有偏向锁、轻量级锁等，但竞争激烈时还是会升级为互斥锁，影响吞吐。

代码示例：

```
// volatile 使用
volatile boolean shutdownRequested;

public void shutdown() {
    shutdownRequested = true;
}

public void doWork() {
    while (!shutdownRequested) {
        // 执行任务
    }
}
```

synchronized 用起来简单，但别滥用，搞不定并发问题就全方法加 synchronized，这在高并发下根本扛不住。像 ConcurrentHashMap 就是用分段锁 + volatile 来减少锁竞争，而不是一把大锁锁住所有操作。

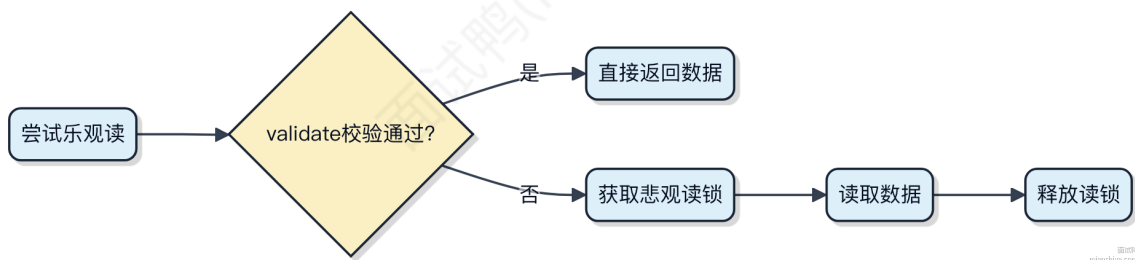
什么是 Java 的 StampedLock?

StampedLock 是 Java 8 引入的一种高性能读写锁，用来替代传统的 ReentrantReadWriteLock。它最大的特点是支持**乐观读**，这在读多写少的场景下能显著提升吞吐量。

ReentrantReadWriteLock 在高并发读时，一旦有线程持有写锁，所有读线程都会被阻塞。而 StampedLock 的乐观读模式允许读操作“无锁”进行，只要数据没被修改就直接返回结果。

使用时需要获取一个 long 类型的 stamp，这个 stamp 代表了锁的状态版本。写操作会改变 stamp，读操作通过校验 stamp 是否被修改来判断数据一致性。

1) 获取乐观读 stamp: `long stamp = lock.tryOptimisticRead();` 2) 执行读逻辑后，必须用 `lock.validate(stamp)` 校验是否失效 3) 如果校验失败，降级为悲观读: `stamp = lock.readLock();`



写锁的获取和释放跟传统方式类似，但写锁不能重入，否则会死锁。这点要特别注意，业务代码里如果嵌套调用容易搞不定。

典型适用场景比如缓存容器、配置中心本地快照，像 Nacos 客户端就用它来保护服务实例列表的读写。压根不经过内核态切换，完全是用户态的 CAS 操作，性能扛得住几千上万 QPS 的读。

不过 API 设计较反人类，try 和 release 必须配对，且 stamp 失效后要手动重试，得包一层工具方法才好用。

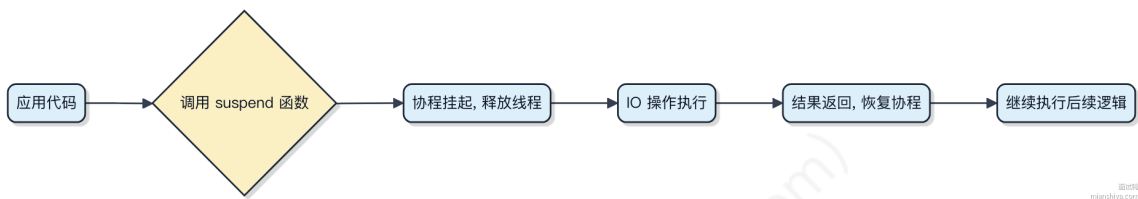
什么是协程？Java 支持协程吗？

协程可以理解成“用户态的轻量级线程”，它让异步代码写起来像同步一样。线程由操作系统调度，而协程由程序自己控制挂起和恢复，切换成本比线程低得多，一个进程里能轻松跑几万个协程。

Java 本身没有原生支持协程。JVM 上的 Kotlin 提供了完整的协程实现，通过 `suspend` 函数和 `launch` 等 API，能在不阻塞线程的情况下处理高并发 IO。比如用 Ktor 或 Spring WebFlux 配合 Kotlin 协程，一个线程就能扛住大量网络请求。

如果非要用 Java 实现类似效果，一般靠响应式编程框架，比如 Project Reactor 或 RxJava。它们用回调机制模拟异步流，虽然也能做到高并发，但代码复杂度比协程高不少。

1) 协程的核心是**挂起函数**，执行到 IO 时自动让出线程，等结果回来再恢复。2) Kotlin 协程底层依赖编译器把 `suspend` 函数拆成状态机，运行时通过 Continuation 管理上下文。3) Java 的虚拟线程 (Virtual Thread) 在某种程度上解决了类似问题。从 JDK19 开始孵化，到 JDK21 正式落地，它由 JVM 调度，能自动利用多核，写法还是传统的同步风格，但每个请求用一个虚拟线程，数量可以上百万。

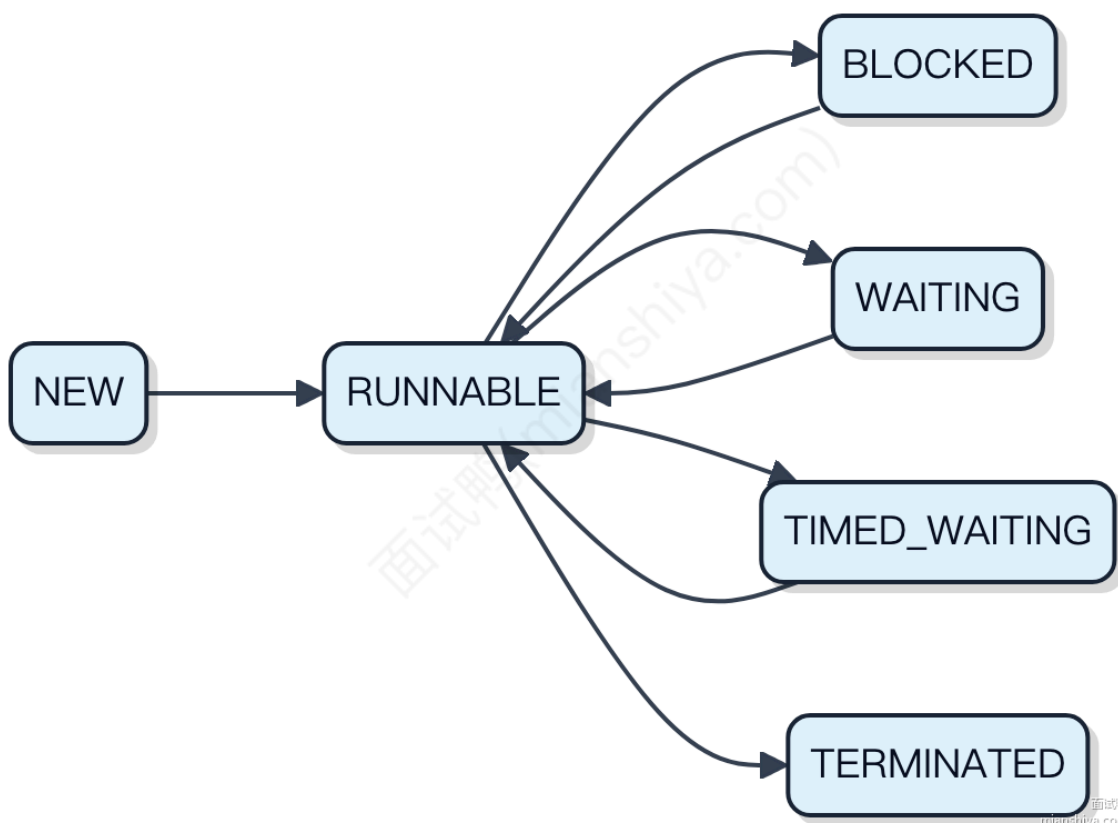


简单说，Java 没有协程，但 Kotlin 有；Java 自己搞了个虚拟线程来解决高并发场景下的线程成本问题，思路不同但目标一致。

线程的生命周期在 Java 中是如何定义的？

线程在 Java 中的生命周期由 `Thread.State` 枚举定义，总共包含 6 种状态，它们反映了线程从创建到终止的完整过程。

- 1) **新建 (NEW)**：线程对象已创建，但还没调用 `start()` 方法，此时线程还没进入运行状态。
- 2) **可运行 (RUNNABLE)**：调用了 `start()` 后，线程进入这个状态。它可能正在运行，也可能在等待 CPU 调度，JVM 不区分就绪和运行中。
- 3) **阻塞 (BLOCKED)**：线程试图获取一个被其他线程持有的监视器锁（synchronized 块/方法），就会进入阻塞状态，直到锁释放。
- 4) **等待 (WAITING)**：线程调用了 `wait()`、`join()` 或 `LockSupport.park()`，会无限期待，直到被其他线程显式唤醒。
- 5) **限期等待 (TIMED_WAITING)**：调用带超时参数的方法，比如 `sleep(long)`、`wait(long)`、`join(long)` 或 `parkNanos`，会在指定时间后自动恢复。
- 6) **终止 (TERMINATED)**：线程任务执行完毕，或者抛出未捕获异常，状态变为终止，此时线程不可复用。



注意，操作系统层面的线程状态和 Java 层不完全一致，JVM 做了一层抽象映射。比如 **WAITING** 状态可能对应内核的休眠，但开发者只需关注 Java 定义的状态流转即可。

Java 中线程之间如何进行通信？

线程通信的本质是协调线程间的执行顺序和数据交换，Java 提供了多种机制来实现这一点。

最基础的方式是通过 **volatile** 关键字修饰变量。它能保证可见性和禁止指令重排，一个线程修改了 **volatile** 变量，另一个线程能立刻看到变化。适合状态标志位的场景，比如用 `volatile boolean running = true` 控制线程循环是否继续。

更复杂的协作依赖 **synchronized** 配合 **wait/notify** 机制。线程在 **synchronized** 块中调用对象的 **wait()** 方法会释放锁并进入等待队列，另一个线程获取到锁后修改状态并调用 **notify()** 或 **notifyAll()**，唤醒等待中的线程。这是生产者-消费者模型的经典实现方式。

```
synchronized (lock) {  
    while (queue.isEmpty()) {  
        lock.wait();  
    }  
    queue.poll();  
    lock.notify();  
}
```

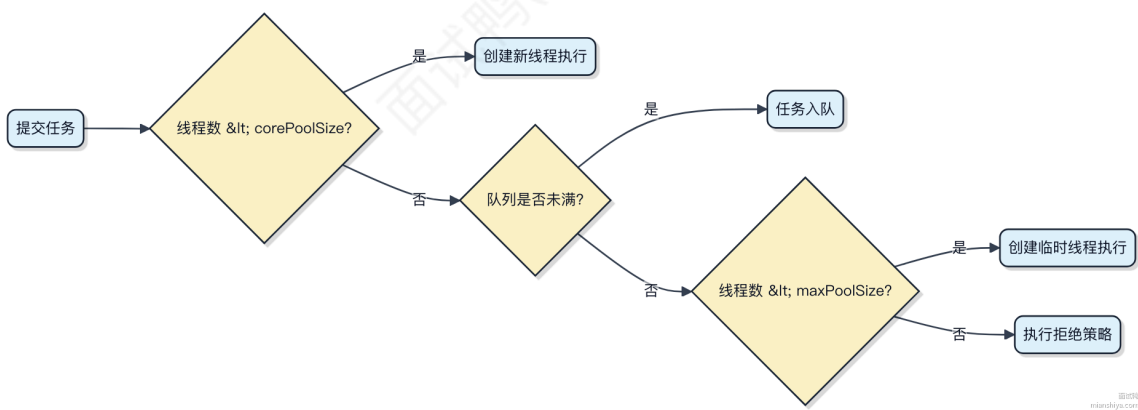
除此之外，JUC 包提供了更高级的工具。比如 **CountDownLatch** 让一个或多个线程等待其他线程完成操作，**CyclicBarrier** 实现线程互相等待到达某个屏障点。像 **ThreadPoolExecutor** 内部就大量使用这些工具协调工作线程。

你了解 Java 线程池的原理吗？

线程池的核心是用少量线程处理大量任务，避免频繁创建销毁线程带来的系统开销。它通过一个阻塞队列暂存待执行任务，核心线程数内的线程会一直存活，超过核心线程数的线程在空闲时会被回收。

ThreadPoolExecutor 的工作流程是这样的：

- 1) 提交任务时，如果当前线程数小于核心线程数，直接创建新线程执行任务，哪怕有空闲线程也一样。
- 2) 如果线程数已达核心线程数，任务会被放入阻塞队列等待。
- 3) 如果队列满了，且线程数小于最大线程数，就会创建临时线程执行任务。
- 4) 如果队列满且线程数达到最大值，就会触发拒绝策略，默认是抛出 RejectedExecutionException。



常见的线程池类型中，**FixedThreadPool** 使用固定大小线程和无界队列，容易导致内存溢出；**CachedThreadPool** 允许创建无限线程，适合短任务但可能压垮系统；**ScheduledThreadPool** 支持定时和周期任务，类似 Timer 但更强大。

实际项目里，Dubbo 和 Tomcat 都自定义了线程池策略。比如 Dubbo 会根据业务场景调整队列长度和拒绝策略，防止雪崩。自己用的时候别直接用 Executors 工厂方法，一定要显式构造 ThreadPoolExecutor，把参数设清楚，不然线上容易搞不定。

如何合理地设置 Java 线程池的线程数？

设置线程数不是拍脑袋决定的，得看任务类型。CPU 密集型和 IO 密集型的策略完全不一样。

CPU 密集型任务，比如大量计算、数据处理，线程太多反而会因为上下文切换拖慢整体性能。这种场景下，线程数一般设为 CPU 核心数 + 1 就够了。JVM 可以通过 `Runtime.getRuntime().availableProcessors()` 拿到核心数。

IO 密集型就不同了，像读写文件、网络请求、数据库操作，线程经常在等 IO 返回，这时候 CPU 是空闲的。为了**压榨 CPU 利用率**，就得开更多线程去“占坑”。常见公式是 $\text{线程数} = \text{CPU 核心数} * (1 + \text{平均等待时间} / \text{平均工作时间})$ 。如果没法精确测算，一个经验做法是设成 $\text{CPU 核心数} * 2$ 或更高。

```
// 示例：基于 CPU 核心数动态设置
int corePoolSize = Runtime.getRuntime().availableProcessors() * 2;
ExecutorService executor = Executors.newFixedThreadPool(corePoolSize);
```

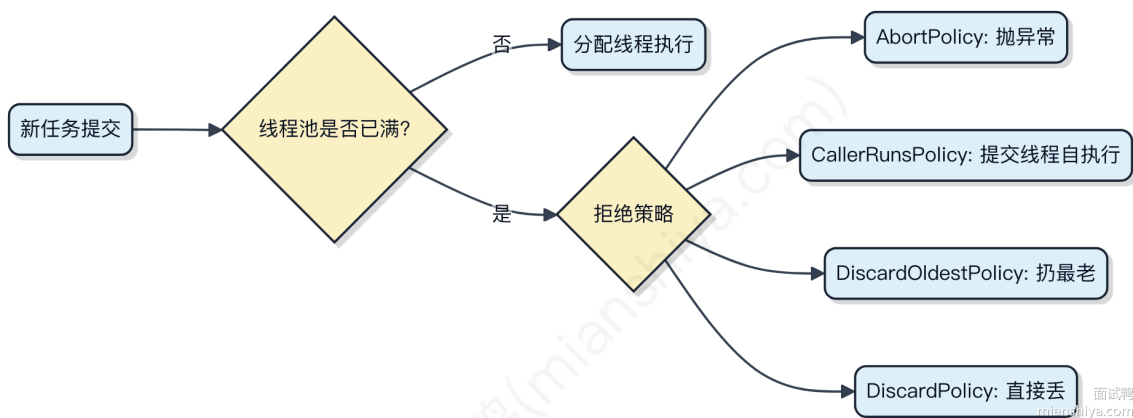

别直接用 Executors 工厂类创建线程池，容易搞出大问题。比如 newFixedThreadPool 用的是无界队列，任务堆积可能 OOM。应该用 ThreadPoolExecutor 手动传参，把核心参数都控制住。

还有种情况是混合型任务，建议拆分，用不同的线程池隔离。比如用 ForkJoinPool 处理并行计算，用自定义线程池跑 HTTP 调用，避免互相影响。

Java 线程池有哪些拒绝策略？

线程池的拒绝策略，说白了就是任务太多，队列也满了，新来的任务怎么处理。JDK 里内置了四种标准方式，每种应对不同的业务需求。

- 1) 直接抛异常。用 AbortPolicy 的时候，一旦线程池饱和，新任务进来立刻抛出 RejectedExecutionException。适合那种不能丢任务、也不能阻塞的场景，比如核心交易下单，扛不住就让调用方立刻知道。
- 2) 谁提交谁处理。CallerRunsPolicy 比较特别，它不会另起线程，而是让提交任务的线程自己去执行这个任务。相当于“你送过来我忙不过来，那你亲自干吧”。这种能减缓任务提交速度，常用于负载较高的服务端，配合 Netty 或 Tomcat 使用效果不错。
- 3) 丢弃最老的任务。DiscardOldestPolicy 会把队列里待得最久的那个任务扔掉，腾位置给新任务。前提是队列得支持优先级或有序性，比如用 LinkedBlockingQueue 就行。适用于实时性要求高、老任务不如新任务重要的场景，像监控数据采集。
- 4) 默默丢弃。DiscardPolicy 最干脆，新任务来了啥也不做，直接丢掉。适合那些允许丢失任务的场景，比如日志上报、埋点数据，丢了几次影响不大。



实际项目中，如果这四种不够用，也可以自定义，实现 RejectedExecutionHandler 接口就行。比如记录日志、写入死信队列，防止任务彻底丢失。

Java 并发库中提供了哪些线程池实现？它们有什么区别？

Java 并发包里的线程池主要靠 ThreadPoolExecutor 来撑场面，它把线程的创建、调度、回收全包了。我们平时用的几种常见线程池，其实都是它通过不同参数组合出来的快捷方式。

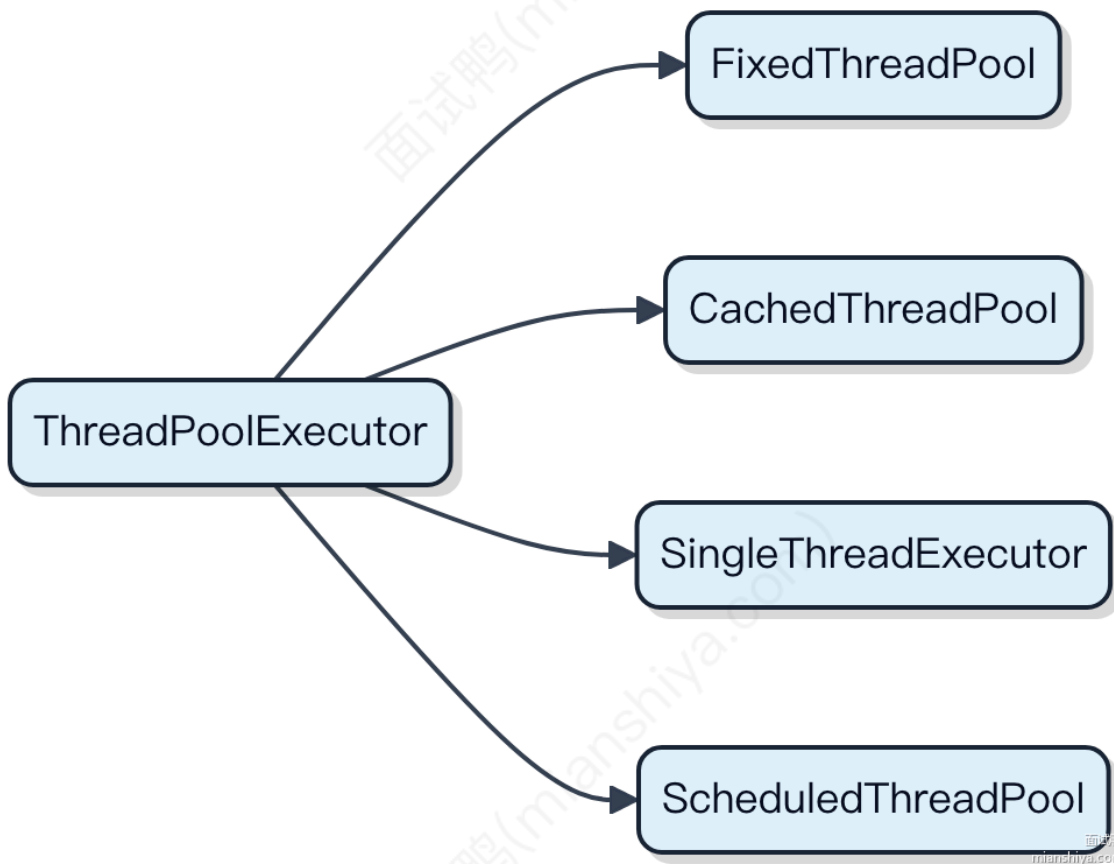
- 1) newFixedThreadPool 创建固定线程数的池子，核心和最大线程数一样，任务多了会放进无界队列排队。适合负载稳定的服务，比如处理大量短平快的请求，但要注意如果任务一直堆积，内存可能扛不住。

2) `newCachedThreadPool` 比较激进，最大线程数是 `Integer.MAX_VALUE`，只要空闲线程不够就新建，60秒没活干就回收。适合突发性强、任务轻的场景，比如处理大量网络回调，但并发一高容易搞垮系统。

3) `newSingleThreadExecutor` 保证任务串行执行，本质是个单线程池，包装后还禁止你重新配置线程数，适合需要顺序处理的任務，比如日志写入。

4) `newScheduledThreadPool` 支持定时和周期性任务，类似 `Timer` 但更强大，能并发执行多个定时任务，`ScheduledExecutorService` 接口定义了 `schedule` 和 `scheduleAtFixedRate` 这些方法。

真正要拿得出手的写法，其实是自己 `new ThreadPoolExecutor`，把核心线程数、最大线程数、空闲时间、队列容量、拒绝策略都明确指定。比如用 `ArrayBlockingQueue` 代替默认的无界队列，配合 `RejectedExecutionHandler` 做降级或打点，这样才能在高并发下稳住。



阿里规约里头不让你用前四种快捷方式，就是因为它们默认用了无界队列或者无限线程，生产环境容易出事。

Java 中的 DelayQueue 和 ScheduledThreadPool 有什么区别？

`DelayQueue` 是一个无界阻塞队列，它里面存放的元素都得实现 `Delayed` 接口，意思是每个任务都有个延迟时间，不到时间你拿不到它。它本身不处理调度逻辑，只是被动地等时间到了才让线程取走任务。

`ScheduledThreadPool` 则是完整的定时调度线程池，基于 `DelayedWorkQueue`（类似 `DelayQueue`）做任务队列，但它自己负责启动线程、执行周期性或延迟任务。你调用 `scheduleAtFixedRate` 或 `scheduleWithFixedDelay`，它就能自动在指定时间触发。

1) `DelayQueue` 需要你自己写消费者线程去 `take` 任务，比如：

```
DelayQueue<DelayedTask> queue = new DelayQueue<>();
while (true) {
    DelayedTask task = queue.take(); // 阻塞等待到期
    task.run();
}
```

2) ScheduledThreadPool 直接帮你封装好了，提交即用：

```
ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);
scheduler.schedule(() -> System.out.println("5秒后执行"), 5, TimeUnit.SECONDS);
```

使用场景上，如果你要做一个轻量级的本地延迟消息队列，比如订单超时关闭，用 DelayQueue 搭配一个消费者就够了。但要做系统级的定时任务调度，比如每10分钟拉一次配置，那就直接上 ScheduledThreadPool。

另外，ScheduledThreadPool 支持 fixed-delay 和 fixed-rate 两种模式，还能取消任务，功能更完整。而 DelayQueue 就是个数据结构，脏活累活都得自己干。

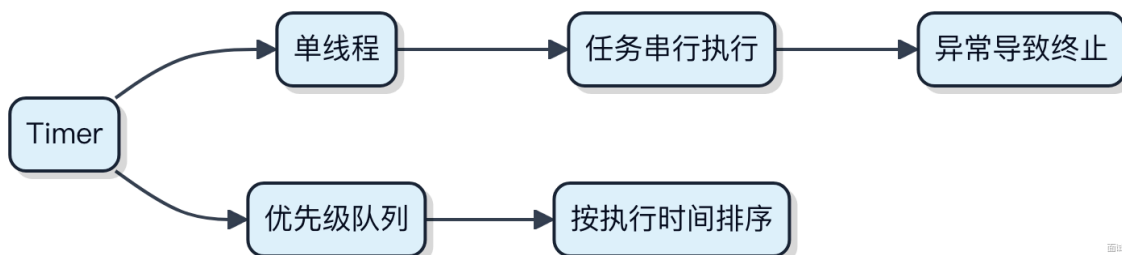
什么是 Java 的 Timer?

Java 的 Timer 是一个内置的任务调度工具，用来在指定时间或周期性地执行任务。它基于单线程模型，内部维护一个 **优先级队列** 来管理待执行的 TimerTask。

- 1) Timer 支持两种调度方式：固定延迟（schedule）和固定速率（scheduleAtFixedRate）。前者保证任务之间的实际间隔至少为设定值，后者尽量让任务按照预定频率执行，即使出现延迟也会尝试追赶。
- 2) 虽然用起来简单，但 Timer 有几个明显短板。一旦任务抛出未捕获异常，整个 Timer 就会挂掉，后续任务全废。而且它只用一个线程跑所有任务，前一个任务阻塞，后面的都得排队等。
- 3) 现在更推荐用 ScheduledExecutorService，比如通过 Executors.newScheduledThreadPool(1) 创建。它支持多线程、能捕获异常、还能更好控制资源。

```
Timer timer = new Timer();
timer.schedule(new TimerTask() {
    public void run() {
        System.out.println("执行一次");
    }
}, 1000); // 1秒后执行
```

对比来看，Timer 适合轻量、简单的场景，比如定时刷新状态。但真正做业务调度，像订单超时、心跳检测这些，一般都会选 Quartz 或 Spring 的 @Scheduled，底层也是基于线程池那套机制。



你使用过哪些 Java 并发工具类？

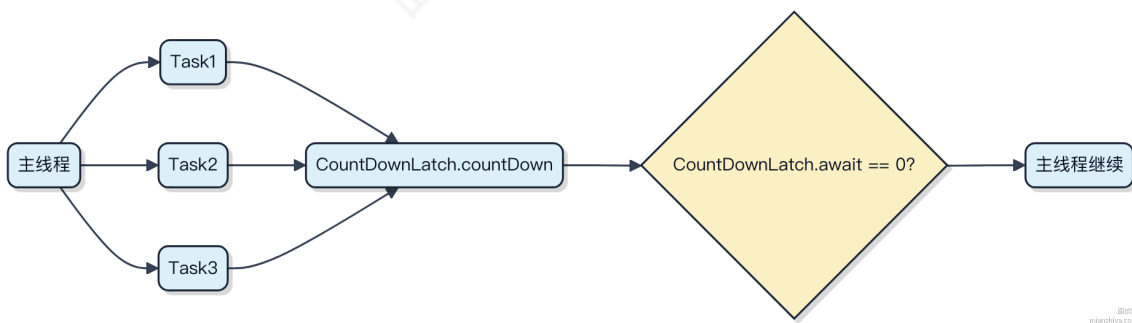
Java 并发包里的工具类，用得最多的是 **JUC** 这一块。日常开发里，几乎离不开这几个场景：线程池管理、并发控制、线程协作。

比如处理批量订单导入，一般会用 `ThreadPoolExecutor` 自定义线程池，核心线程数根据 CPU 核心来定，IO 密集型任务通常设为 $2 * \text{CPU 数}$ 。拒绝策略选 `RejectedExecutionHandler` 的时候，用 `CallerRunsPolicy` 能让主线程自己执行，避免雪崩。

要控制并发读写，`ReentrantReadWriteLock` 就很实用。像缓存系统这种读多写少的场景，多个读线程可以同时进入，写的时候才独占，性能比直接用 `synchronized` 高不少。

信号量 `Semaphore` 适合做限流。假设调用第三方支付接口，QPS 上限是 100，那就初始化一个 permit 数为 100 的 `Semaphore`，每次请求前 `acquire`，结束后 `release`，简单又有效。

还有 `CountDownLatch` 和 `CyclicBarrier`，虽然都用来协调线程，但用途不一样。前者是一等多，比如主线程等 3 个加载任务都完成再启动服务；后者是多等一，所有线程互相等待，到齐了才继续，像并发压测时统一发压。



`ConcurrentHashMap` 更是高频，缓存计数、共享状态存储都靠它。和 `Collections.synchronizedMap` 不同，它分段锁，1.8 后直接用 CAS + `synchronized` 控制桶，冲突少，吞吐高。

这些工具类组合起来，基本能搞定大多数并发场景的脏活累活。

什么是 Java 的 Semaphore？

信号量（Semaphore）本质是个计数器，用来控制同时访问共享资源的线程数量。你可以把它想象成一组通行证，线程必须拿到许可才能执行，用完归还。

1) 初始化时指定许可总数，比如 `new Semaphore(3)` 就是最多 3 个线程能同时访问。2) `acquire()` 方法会阻塞线程直到有空闲许可，`release()` 把许可还回去。3) 特别适合做流量控制，比如限制数据库连接池的并发连接数不超过 10 个，用它就非常合适。

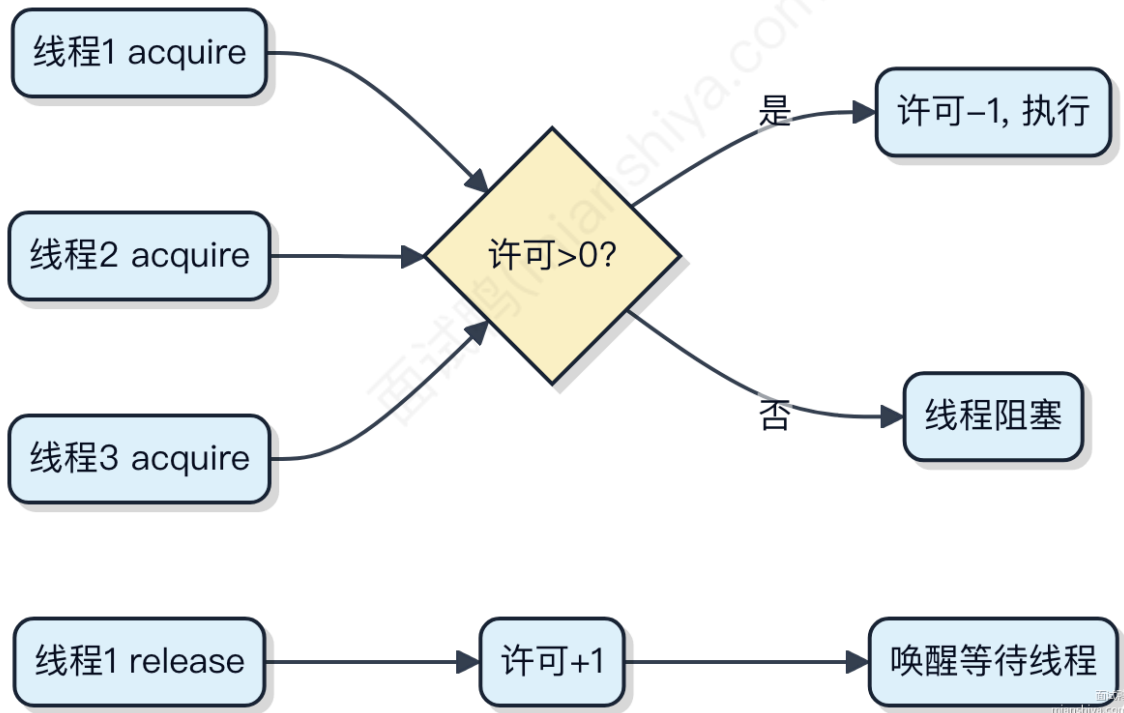
和 `synchronized` 或 `ReentrantLock` 不一样，**Semaphore** 可以允许多个线程同时进入临界区，不是互斥锁，而是限流阀。比如用在秒杀系统里，控制每秒最多 100 个请求进入处理，后面的得等着，避免后端被压垮。

代码上很简单：

```
Semaphore semaphore = new Semaphore(3);
semaphore.acquire(); // 获取许可
try {
    // 执行操作
} finally {
```

```
semaphore.release(); // 一定要释放  
}
```

漏掉 release 就会导致许可越来越少，最后所有线程都被卡住。所以通常放在 finally 块里确保执行。



什么是 Java 的 CountDownLatch?

CountDownLatch 的本质是一个一次性使用的同步工具，用来让一个或多个线程等待其他线程完成一组操作。它内部维护一个计数器，初始化时指定需要等待的事件数量。

1) 当某个线程调用 `countDown()` 方法，计数器就减 1，这个操作是线程安全的，而且不会阻塞调用线程。2) 而其他线程可以调用 `await()` 方法来阻塞自己，直到计数器归零才会继续执行。3) 一旦计数器归零，所有等待的线程会被释放，后续再调用 `await` 也会直接通过，因为这个 latch 已经被触发了。

比如在 Web 应用启动时，主线程要等 3 个数据加载线程都完成后才开始对外提供服务，就可以初始化一个 `CountDownLatch(3)`，每个加载线程完成任务后调 `countDown`，主线程在 `await` 上等着就行。

代码上很简单：

```
CountDownLatch latch = new CountDownLatch(3);  
// 子线程中  
latch.countDown();  
  
// 主线程中  
latch.await(); // 阻塞直到计数归零
```

注意它是一次性的，不能重置。如果需要重复使用，得换 `CyclicBarrier` 或者重新 new 一个实例。它也不保证线程安全的业务逻辑，只是控制执行时机，别把它当成数据同步手段。

如何在 Java 中控制多个线程的执行顺序？

线程执行顺序的控制本质上是对线程间协作的管理，Java 提供了多种手段来实现这一点，关键在于理解线程间的**等待-通知机制**和**状态依赖**。

1) 使用 `join()` 是最直接的方式。主线程启动 t1 后调用 `t1.join()`，就会阻塞直到 t1 执行完，接着启动 t2 并 join，自然形成 `t1 → t2 → main` 的顺序。

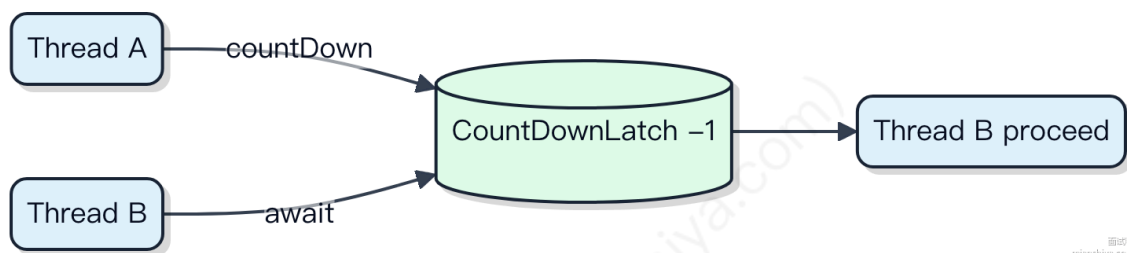
```
Thread t1 = new Thread(() -> System.out.println("T1"));
Thread t2 = new Thread(() -> System.out.println("T2"));
t1.start();
t1.join();
t2.start();
t2.join();
```

2) `CountDownLatch` 更适合多线程协同场景。比如让 3 个线程按 A→B→C 执行，可以设两个门，A 完成后减一，B 等待第一个门，以此类推。它的好处是不绑定具体线程，灵活度高。

3) `CyclicBarrier` 适合让多个线程互相等待到达某个点后再一起出发，比如并发测试时统一发压。

4) 通过 `synchronized` 配合 `wait/notify` 也能手动控制，但容易出错，一般推荐用 JUC 工具类。

其实大多数情况下，`join()` 和 `CountDownLatch` 就能解决 90% 的需求。像 `CompletableFuture` 也可以通过链式调用控制异步任务顺序，适合更复杂的编排场景。



如何优化 Java 中的锁的使用？

锁这玩意儿用不好就是性能瓶颈，尤其在高并发场景下。优化的关键是减少争用、降低开销，而不是一上来就 `synchronized` 啥都包住。

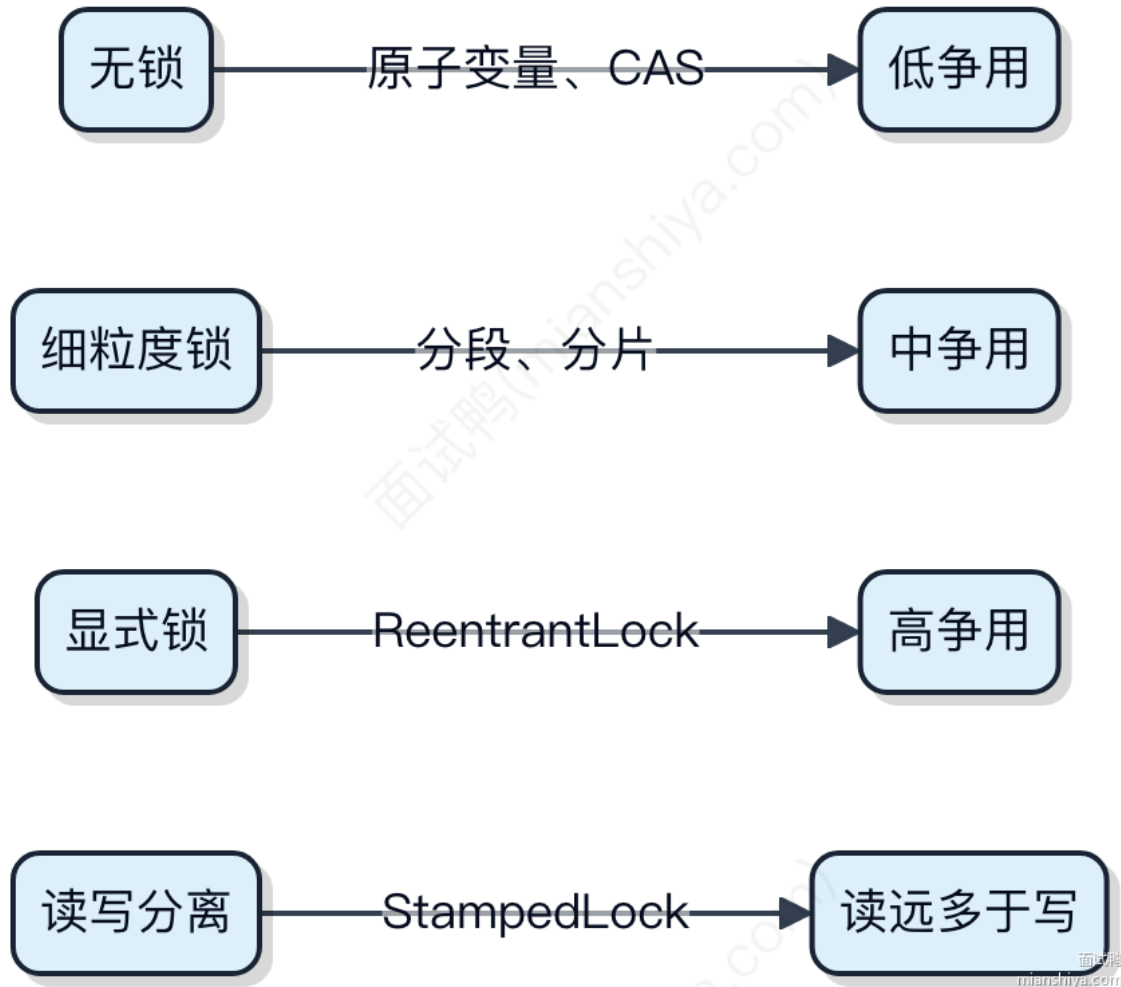
1) 能不用锁就不用，优先考虑无锁结构。比如计数用 `AtomicInteger`，状态机切换用 `AtomicReference`，这些底层靠 CAS 实现，比 `synchronized` 轻太多。像 Kafka、Netty 里大量用原子类扛流量，压根不经过内核态。

2) 缩小锁粒度，别一把锁管全场。`ConcurrentHashMap` 就是个例子，它把数据分段，每个 segment 独立加锁，读写不同桶根本互不干扰。你也可以拆成多个锁对象，按业务维度隔离，比如按用户 ID 取模分片加锁。

3) 替换重锁。`synchronized` 在 JDK 1.6 后其实做了很多优化，偏向锁、轻量级锁能自旋抢，但竞争激烈时还是会膨胀成重量级锁。这时候用 `ReentrantLock` 更灵活，支持公平锁、可中断、超时获取，还能配合 `Condition` 实现精准通知。不过要注意 `finally` 里 `unlock`，漏了会死锁。

4) 读多写少用读写锁。`ReentrantReadWriteLock` 允许多个读线程并发，写线程独占。但要注意“写饥饿”，大量读请求可能让写线程一直抢不到锁。JDK 8 引入的 `StampedLock` 支持乐观读，性能更好，但使用更复杂，得小心。

版本校验。



什么是 Java 中的锁自适应自旋？

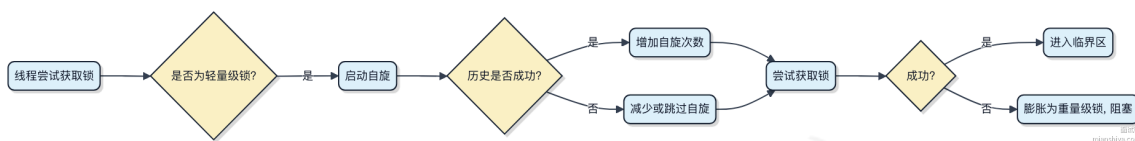
自旋锁大家都知道，就是线程不立刻阻塞，而是跑个空循环看看持有锁的线程很快会不会释放。早期的自旋是固定的，比如就自旋10次，次数写死，效果不好——锁持有一百毫秒你还自旋？纯浪费CPU。

后来 JVM 做了优化，引入**自适应自旋**。它的意思很简单：自旋的次数和时间不再固定，而是根据**同一把锁的历史表现**来动态调整。如果之前自旋成功过，这次就多自旋几轮；如果总是失败，下次就少自旋甚至直接不自旋。

这玩意对 `synchronized` 的重量级锁特别有用。在锁竞争不激烈、持有时间短的场景下，比如用 `ReentrantLock` 或 `synchronized` 在临界区做点简单操作，自适应机制能让等待线程快速接棒，避免频繁陷入内核态的线程挂起和恢复，开销能从微秒级降到纳秒级。

代码层面你不用做什么，这是 HotSpot 虚拟机在背后自动做的。但你要知道，在高并发且临界区短的场景里，`synchronized` 并没有想象中那么“重”，一部分功劳就得算在自适应自旋头上。

比如在 JUC 里很多原子类的操作，或者 `ConcurrentHashMap` 的桶级锁，都有可能触发这种优化。你看到的是一个 `synchronized`，背后可能是自旋 + 偏向 + 轻量锁升级的一整套组合拳。



你使用过 Java 中的哪些原子类？

Java 的原子类在并发编程里用得挺多，尤其是要保证线程安全又不想加锁的时候。这些类底层靠的是 **CAS** 操作，配合 `volatile` 和 `Unsafe` 类实现高效无锁并发。

比如最常用的是 `AtomicInteger`，用来做计数器特别合适。你调它的 `incrementAndGet()` 方法，其实就是原子性地加 1，不用 `synchronized` 也能保证线程安全。

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // 线程安全的 ++i
```

除了整型，还有 `AtomicLong`、`AtomicBoolean`，用法差不多。如果要操作引用类型，可以用 `AtomicReference`，它能保证引用的读写是原子的。

处理数组的话，有 `AtomicIntegerArray`，你可以对数组里的某个位置做原子更新，其他元素不受影响。

还有一类是带“累加器”语义的，比如 `LongAdder`。它跟 `AtomicLong` 比，在高并发下性能更好。因为 `AtomicLong` 在竞争激烈时 CAS 会重试很多次，而 `LongAdder` 用了分段累加的思想，最后再汇总，把冲突分散了。

```
LongAdder adder = new LongAdder();
adder.add(10); // 高并发下比 AtomicLong 更高效
```

像 `ConcurrentHashMap` 的计数场景就适合用 `LongAdder`。另外 `AtomicStampedReference` 和 `AtomicMarkableReference` 是为了解决 ABA 问题设计的，给引用带上版本号或者标记位。

总的来说，低并发用 `AtomicInteger` 这种就行，高并发场景优先考虑 `LongAdder` 或 `DoubleAdder`。

你使用过 Java 的累加器吗？

Java 的并发累加器，主要是 `LongAdder` 和 `DoubleAdder`，这玩意儿在高并发计数场景下比 `AtomicLong` 能扛得多。

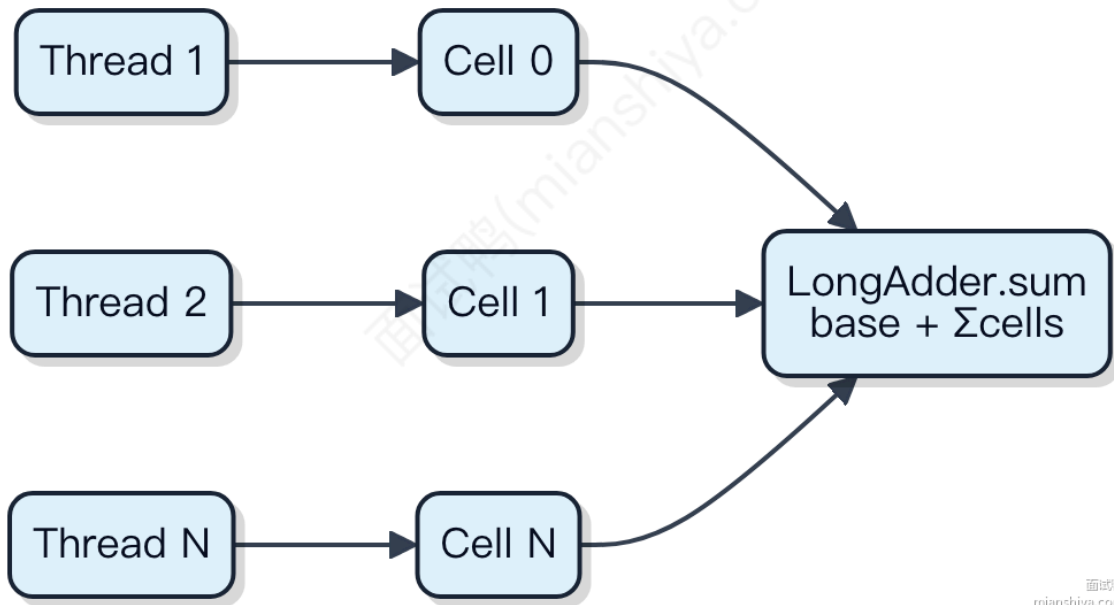
它背后的思路很直接：当多个线程同时更新一个值时，`AtomicLong` 会因为 CAS 竞争导致大量线程自旋失败，性能急剧下降。而 `LongAdder` 采用分段累加的思想，把冲突分散到多个 cell 中，每个线程尽量操作自己的 cell，最后再把所有 cell 加起来。这种空间换时间的策略，在 100+ 线程并发更新时，吞吐量能高出一个数量级。

适合用它的场景比如监控埋点、请求计数、分布式 ID 生成器中的本地步长控制。像 `Micrometer` 这种监控库底层就用了 `LongAdder` 来收集指标。

```
LongAdder adder = new LongAdder();
adder.increment(); // 线程安全累加
adder.sum();       // 获取总和（可能有延迟）
```


注意 `sum()` 不是强一致的，它返回的是调用时刻的快照，中间可能有正在更新的 cell 没合并进来。如果需要严格实时总数，还是得用 `AtomicLong`，但你要清楚这是以性能为代价的。

- 1) 不要频繁调用 `reset()`，尤其是在并发环境下，可能导致统计丢失
- 2) 读多写少的场景没必要用 `LongAdder`，`AtomicLong` 更轻量
- 3) `DoubleAdder` 同理，只是对应浮点类型

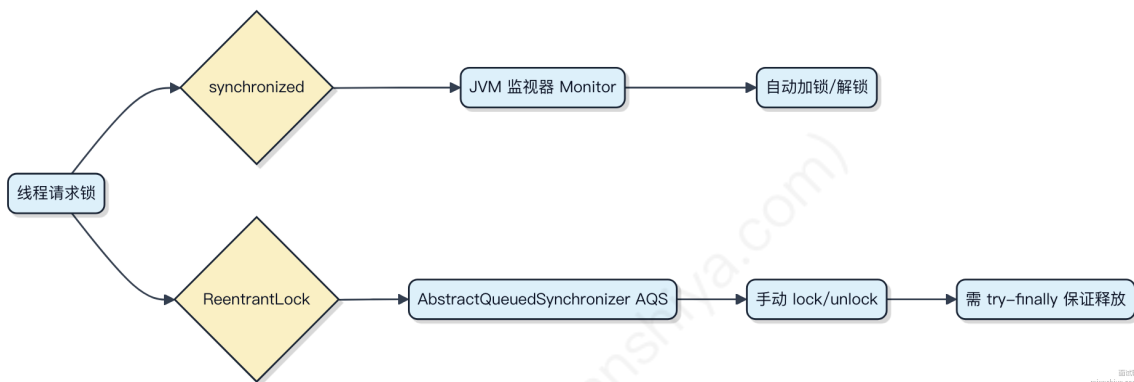


面试鸭
mianshiya.com

Synchronized 和 ReentrantLock 有什么区别？

`synchronized` 是 JVM 层面实现的互斥锁，代码进入同步块时自动加锁，退出时自动释放，哪怕抛异常也不会死锁。`ReentrantLock` 是 JDK 提供的 API 级别可重入锁，必须手动调用 `lock()` 和 `unlock()`，通常要配合 `try-finally` 使用。

- 1) 等待可中断：`ReentrantLock` 能响应中断，比如线程 A 拿着锁不放，B 等太久可以主动中断放弃，避免无限等下去。`synchronized` 只能傻等。
- 2) 公平锁支持：`ReentrantLock` 可以通过构造函数指定是否使用公平模式，按排队顺序获取锁。`synchronized` 始终是非公平的，谁抢到算谁的。
- 3) 条件等待：`ReentrantLock` 配合 `Condition` 接口能实现更灵活的线程通信，比如生产者消费者模型里可以分别控制等待和唤醒。`synchronized` 只能靠 `wait/notify`，灵活性差一些。
- 4) 性能上早期差距明显，但自从 `synchronized` 加了偏向锁、轻量级锁优化后，大多数场景下性能已经和 `ReentrantLock` 差不多，甚至更好。

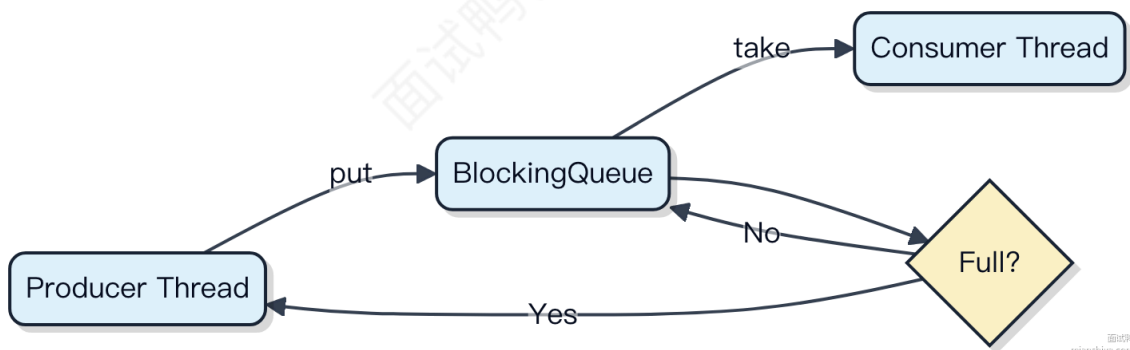


一般业务开发优先用 `synchronized`，足够简单安全。只有在需要超时尝试、可中断、公平锁这些高级特性时，才选 `ReentrantLock`。

你使用过 Java 中的哪些阻塞队列？

Java 里的阻塞队列其实就那么几个常用选手，搞懂它们的特性基本就能应对大部分并发场景。

- 1) **ArrayBlockingQueue** 是个有界队列，底层用数组实现，构造时必须指定容量。一旦满了，生产者线程就会被阻塞；空了，消费者线程也会挂起。适合做资源池或者任务队列这种需要限流的场景，比如 Tomcat 的工作线程队列就可以用它。
- 2) **LinkedBlockingQueue** 基于链表，可以有界也可以无界（默认 `Integer.MAX_VALUE`）。读写操作分别用两把锁控制，吞吐量比 `ArrayBlockingQueue` 高一些。像 `ThreadPoolExecutor` 里的默认队列就是它，尤其是 `Executors.newFixedThreadPool()` 返回的那种线程池。
- 3) **SynchronousQueue** 比较特别，它不存元素，每个插入操作得等到另一个线程的移除操作。相当于直接“手递手”交接，所以生产者和消费者必须同时到场。`Cached` 线程池就用它，任务来了立马找线程处理，没有闲置队列堆积。
- 4) **DelayQueue** 存放实现了 `Delayed` 接口的对象，只有到期才能取。常用于定时任务调度，比如 Netty 的 `HashedWheelTimer` 底层可以用它来管理超时任务。
- 5) **PriorityBlockingQueue** 是个无界优先级队列，按优先级出队，内部用堆实现。适合需要优先处理高优先级任务的场景，比如消息中间件里的优先级消息。



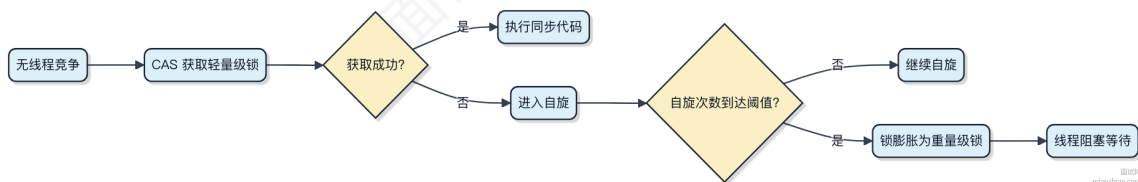
选哪个看业务需求：要限流用 `Array`，高吞吐考虑 `Linked`，要即时传递上 `Synchronous`，带延迟或优先级的再选对应的特殊队列。

Java 中的 synchronized 轻量级锁是否会进行自旋?

轻量级锁的设计初衷是为了在**没有多线程竞争或竞争不激烈**的场景下，减少传统重量级锁带来的操作系统互斥量（mutex）开销。它确实会尝试自旋，但关键得看 JVM 的具体实现和当前锁的状态。

- 1) 当一个线程尝试获取轻量级锁时，JVM 会在对象头中使用 CAS 操作尝试将对象的 Mark Word 替换为指向线程栈中锁记录的指针。如果这个操作失败了，说明有冲突，这时候 JVM 不会立刻膨胀为重量级锁。
- 2) 此时，JVM 会进入**自旋等待**阶段，也就是让当前线程空转几个 CPU 周期，看看持有锁的线程能不能快速释放。这种策略在多核 CPU 上特别有用，因为上下文切换的成本远高于短暂的自旋。
- 3) 但如果自旋一定次数后（比如 10-60 次，具体由 JVM 参数 `-XX:PreBlockSpin` 控制，现代 HotSpot 已改为自适应），锁还没被释放，JVM 就会认为竞争激烈，将轻量级锁**膨胀为重量级锁**，后续线程就只能阻塞等待。

所以，轻量级锁本身不是“一直自旋”，而是**带有自旋优化的过渡机制**。它的整个流程是：无锁 → 轻量级锁（CAS + 短暂自旋）→ 自旋失败则升级为重量级锁。



Java 的 synchronized 是怎么实现的?

synchronized 的底层其实依赖 JVM 对 **monitor** 的支持，每个对象都有一个与之关联的 monitor。线程要进入 synchronized 代码块，必须先获取这个 monitor 的所有权。

monitor 在 HotSpot 虚拟机里是用 C++ 实现的 ObjectMonitor 类，它的竞争机制不是一上来就重量级加锁。而是从**无锁状态**开始，随着竞争情况升级：

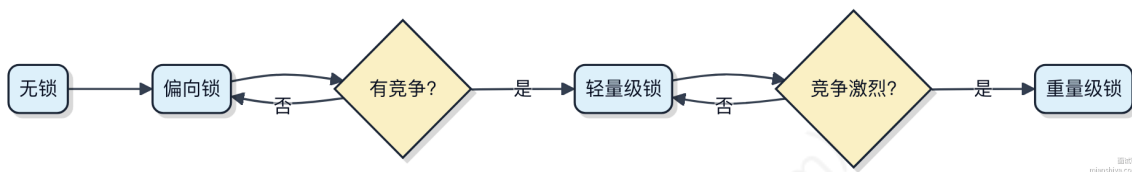
- 1) 没有竞争时，使用 **偏向锁**，记录下线程 ID，下次同一个线程进来直接通过，连 CAS 都省了
- 2) 出现竞争但不激烈，升级为 **轻量级锁**，通过 CAS 尝试把对象头的 Mark Word 替换为指向线程栈中锁记录的指针
- 3) 竞争激烈时，膨胀为 **重量级锁**，这时候线程会进入阻塞态，由操作系统来调度，性能开销大

锁的状态信息都存在对象头的 Mark Word 里，64 位虚拟机下占 8 字节，里面存了哈希码、分代年龄、锁标志位等。

```
synchronized (obj) {  
    // 字节码层面会生成 monitorenter 和 monitorexit 指令  
    // 对应到 ObjectMonitor 的 enter() 和 exit()  
}
```

解锁时要释放 monitor，并唤醒等待的线程。如果多个线程同时竞争，可能触发锁膨胀甚至全局停顿。

整个过程是 JVM 自动管理的，开发者不用干预，这也是为什么叫“内置锁”。

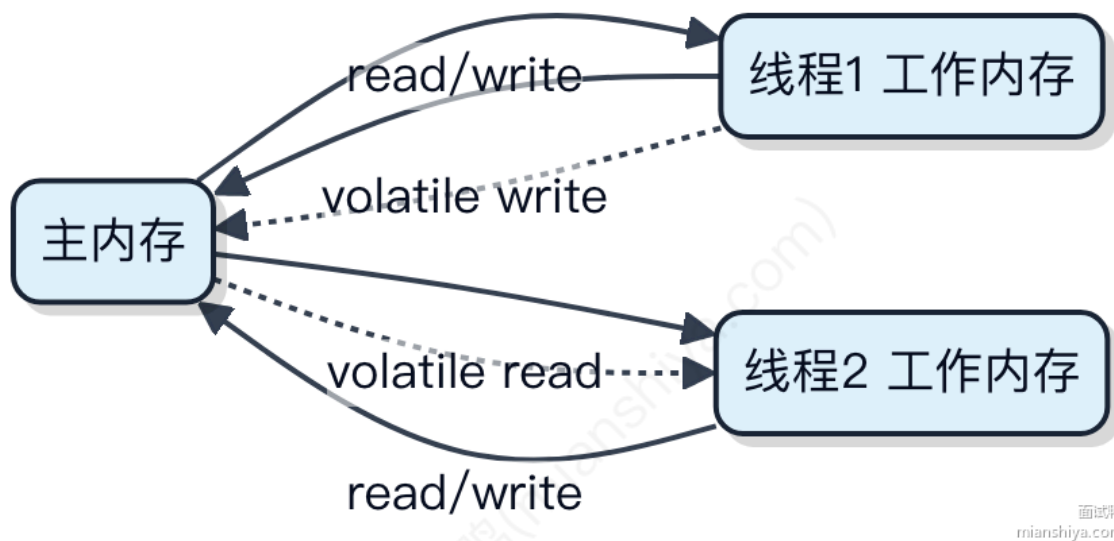


什么是 Java 内存模型（JMM）？

Java 内存模型（JMM）不是描述内存分布，而是定义多线程环境下变量的可见性、原子性和有序性规则。它屏蔽了硬件和操作系统的差异，确保程序在不同平台下对内存的访问行为一致。

每个线程有自己的工作内存，里面保存了主内存中共享变量的副本。线程对变量的操作都在工作内存里进行，修改后要写回主内存才能被其他线程看到。这就可能引发可见性问题，比如一个线程改了变量，另一个线程压根不经过主内存，读到的就是旧值。

- 1) 原子性：基本读写是原子的，但像 `i++` 这种复合操作就不行，得靠 `synchronized` 或 `volatile` 配合 CAS 来保证。
- 2) 可见性：一个线程修改了变量，其他线程能立即知道。`volatile` 变量写会强制刷回主内存，读则强制从主内存加载。
- 3) 有序性：编译器和处理器可能会重排序指令来优化性能，但 `happens-before` 规则限制了哪些重排是合法的。比如 `volatile` 写一定发生在后续的 `volatile` 读之前。



典型场景如双重检查单例，`instance` 不加 `volatile`，就可能因为重排序导致返回未初始化完的对象。用 `synchronized` 能解决，但 `volatile` 更轻量。

什么是 Java 中的原子性、可见性和有序性？

多线程环境下，这三个特性是理解并发问题的根基。

原子性指的是一个操作要么全部执行成功，要么完全不执行，不会被线程调度打断。比如 `i++` 实际上是读、改、写三步，不具备原子性。用 `AtomicInteger` 的 `incrementAndGet()` 就能保证这一步是原子的，底层靠的是 CPU 的 CAS 指令。

可见性是指当一个线程修改了共享变量的值，其他线程能立即知道这个变化。没有同步手段时，线程可能一直用着自己工作内存里的旧值。`volatile` 关键字能解决这个问题，它会强制线程从主内存读写变量。`synchronized` 和 `final` 也能保证可见性。

有序性涉及到指令重排的问题。编译器和处理器为了优化性能，可能会改变代码执行顺序。但在多线程下，这种重排可能导致意料之外的结果。`volatile` 能禁止特定类型的重排序，`synchronized` 则通过同一时刻只允许一个线程进入同步块来保证有序。



比如在双重检查单例里，`instance` 字段必须加 `volatile`，否则可能发生返回了一个未完全初始化的对象。重排序可能导致对象还没构造完，引用就已经指向了那块内存。

什么是 Java 的 happens-before 规则？

Java 的 happens-before 规则是用来定义多线程环境下操作可见性的一套规则。它不等于时间先后，而是一种**先行发生关系**，保证一个操作的结果对另一个操作可见。

比如线程 A 修改了某个变量，线程 B 读到了这个修改，这就构成了 happens-before 关系。如果没有这种规则约束，由于指令重排、缓存不一致等问题，B 可能压根看不到 A 的改动。

常见的 happens-before 规则有这么几种：

1) 同一个线程中的操作，按照代码顺序自然形成 happens-before。2) `volatile` 变量的写操作 happens-before 之后对该变量的读。3) `synchronized` 块的解锁 happens-before 后续对同一锁的加锁。4) 线程的 `start()` 调用 happens-before 新线程里的任意操作。5) 线程的所有操作 happens-before 对该线程的 `join()` 返回。6) 传递性：A happens-before B，B happens-before C，则 A happens-before C。

举个例子，用 `volatile` 修饰状态标志位，就能确保启动线程前的初始化操作不会被重排到写标志位之后，避免其他线程看到未完成初始化的状态。

```
volatile boolean initialized = false;

// 线程 A
data = 1;
initialized = true; // 写 volatile

// 线程 B
if (initialized) { // 读 volatile
    System.out.println(data); // 能安全看到 data = 1
}
```

这套规则其实是 JMM（Java 内存模型）的一部分，编译器和 CPU 的优化都得遵守它，才能让并发编程有据可依。

什么是 Java 中的指令重排？

Java 代码执行时，JVM 和 CPU 为了优化性能，可能会调整指令的执行顺序，这就是**指令重排**。只要结果和程序顺序一致，这种重排就不会被察觉，但多线程环境下就容易出问题。

- 1) 编译器会在生成字节码时重排，比如把循环不变量提到外面。
- 2) CPU 在运行时也会根据流水线效率动态调度指令，比如先执行不依赖前面结果的独立操作。
- 3) 内存系统还有自己的缓冲和写合并机制，看起来也像指令乱序。

最典型的坑是双重检查单例：

```
if (instance == null) {
    synchronized (Singleton.class) {
        if (instance == null) {
            instance = new Singleton(); // 这一步可能被重排
        }
    }
}
```

`new Singleton()` 实际分三步：分配内存、初始化对象、引用赋值。如果 3 在 2 前面执行，另一个线程可能拿到未初始化完的对象。

解决办法是用 `volatile` 修饰变量，它能禁止特定类型的指令重排，保证可见性和有序性。或者直接用静态内部类实现单例，更简洁安全。

Java 线程池核心线程数在运行过程中能修改吗？如何修改？

线程池的核心线程数在运行时是可以修改的，Java 提供了直接的方法来动态调整。关键在于

`ThreadPoolExecutor` 的 `setCorePoolSize` 方法。

- 1) 调用 `setCorePoolSize(int corePoolSize)` 可以实时修改核心线程数。如果新值大于当前值，线程池会尝试创建新线程来达到目标，前提是队列中有等待任务。如果小于当前值，超出部分的空闲线程会在下次空闲时被回收。
- 2) 这个机制让线程池具备弹性伸缩能力，适合负载波动大的场景。比如在电商大促期间，可以通过监控 QPS 动态上调核心线程数，扛住突发流量；高峰过去再降下来，避免资源浪费。
- 3) 代码上很简单：

```
executor.setCorePoolSize(20);
```

一行代码就能生效，不需要重启或重建线程池。

这种动态调整能力，是 `ThreadPoolExecutor` 区别于固定大小线程池的关键优势。很多中间件比如 **Dubbo**、**Tomcat** 的工作线程池都基于此实现运行时调参。

不过要注意，频繁修改可能引发线程震荡，建议结合监控系统做平滑调整。

当 Java 的 `synchronized` 升级到重量级锁后，所有线程都释放锁了，此时它还是重量级锁吗？

重量级锁一旦升级，就不会降级。`synchronized` 的锁状态从无锁 -> 偏向锁 -> 轻量级锁 -> 重量级锁，这个过程是单向的。

1) 当多个线程竞争同一个对象锁且出现阻塞时，JVM 会将锁膨胀为**重量级锁**，这时候依赖操作系统互斥量（mutex）来实现线程阻塞和唤醒，代价比较高。

2) 即使所有等待线程都释放了锁，甚至当前没有线程持有它，这个对象的 mark word 仍然会保持重量级锁的标记。下一次有线程再去争用，直接走重量级锁流程，不会重新尝试偏向或轻量级。

3) 也就是说，锁升级是永久性的，直到对象被 GC 回收为止。这也是为什么高并发场景下过度使用 synchronized 可能带来性能问题——压根不经过轻量级路径优化了。

举个例子，像 ConcurrentHashMap 在 JDK 8 之前用 segment 分段锁就是为了避免大范围 synchronized，就是怕搞不定这种不可逆的锁膨胀。

代码层面你看不到直接控制，这是 JVM 自动决策的：

```
synchronized (obj) {  
    // 多个线程进来抢，有人挂起  
    // 此处触发锁膨胀后，以后永远是重量级  
}
```

所以设计系统时，如果能用 CAS 或 ReentrantLock 控制锁粒度，很多时候更灵活。毕竟 synchronized 的自动升级机制虽然省心，但一旦上去就下不来了。

你了解时间轮（Time Wheel）吗？有哪些应用场景？

时间轮本质是个环形队列，用数组或链表实现，每个槽代表一个时间间隔。指针按固定频率转动，到期任务被触发。它的核心优势是**调度大量定时任务时效率高**，插入和删除操作都是 O(1)。

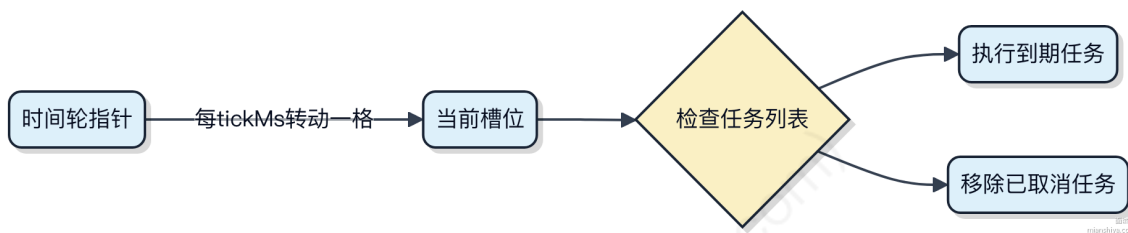
适合用在需要频繁创建、取消定时器的场景。比如 Netty 里做连接空闲检测，Kafka 用它处理延迟消息和重试机制。像 Redisson 分布式锁续期也是靠类似机制撑住几万并发锁的自动刷新。

相比 JDK 的 Timer 或 ScheduledThreadPoolExecutor，时间轮在海量任务下更稳。后者底层是小顶堆，每次增删都要 log(n)，任务一多就容易卡。而时间轮把时间分片，任务挂到对应槽上，压根不经过复杂排序。

代码层面看，最简模型长这样：

```
class SimpleTimeWheel {  
    private Task[] buckets;  
    private int tickMs, wheelSize;  
    private long currentTime;  
  
    public void addTask(Task task) {  
        int idx = (task.delayMs / tickMs) % wheelSize;  
        buckets[idx].add(task);  
    }  
}
```

实际用的时候会加多级时间轮，像 Kafka 那样搞层级化推进，既能支持秒级精度又能覆盖几天后的任务。



什么是 Java 中的 ABA 问题？

ABA 问题出现在使用 CAS (Compare-And-Swap) 实现的无锁并发结构中，比如 `AtomicInteger` 或 `ConcurrentLinkedQueue`。它不是说值变了又变回来这么简单，而是 **CAS 操作无法感知中间是否发生过变化**。

举个例子：线程1读取某个变量值为 A，然后去执行 CAS，预期还是 A 就改成 B。但在这期间，其他线程可能把 A 改成 B 又改回 A。对线程1来说，值没变，于是顺利提交。可实际上系统状态已经不同了，比如资源被释放又分配，导致悬挂指针或重复释放等问题。

这个问题在操作系统或内存管理里尤其危险，但在 Java 里一般通过 **版本号机制** 解决。典型方案是用 `AtomicStampedReference`，它不仅存对象引用，还带一个整型的 stamp，每次修改递增。这样即使值从 A→B→A，stamp 也从 0→1→2，线程能发现版本不对，拒绝更新。

所以，单纯靠值相等来判断“没变”是不够的，得加上时间维度或者版本号。像 Redis 的乐观锁、数据库的 MVCC 其实也面临类似问题，解决思路一脉相承。

什么是 Java 的 CyclicBarrier？

CyclicBarrier 是一个让一组线程互相等待的同步工具，直到所有线程都到达某个屏障点 (barrier point)，再一起继续执行。它特别适用于多线程并行计算中需要分阶段协同的场景。

比如用 Fork/Join 框架做大数据处理时，每个线程处理完自己的那块数据后，得等所有人汇总结果才能进入下一步，这时候 CyclicBarrier 就能派上用场。

它的构造可以指定参与的线程数，比如 `new CyclicBarrier(3)` 表示要等 3 个线程都调用 `await()` 才会放行。而且和 `CountDownLatch` 不一样，它是可重用的，一次“拦停”结束后可以重置状态，下次还能用。

- 1) 调用 `await()` 的线程会被阻塞，直到数量达标
- 2) 最后一个到达的线程会触发 `barrierAction` (可选的 `Runnable`)
- 3) 如果中断其中一个线程，整个 barrier 会打破，其他线程会抛出 `BrokenBarrierException`

代码上看很简单：

```
CyclicBarrier barrier = new CyclicBarrier(2);
new Thread(() -> {
    System.out.println("线程1准备就绪");
    barrier.await();
    System.out.println("线程1继续执行");
}).start();

Thread.sleep(100);
System.out.println("主线程准备就绪");
```



```
barrier.await();
System.out.println("主线程继续执行");
```

注意别把它和 CountDownLatch 搞混了。后者是一次性的，且是“一个线程等一堆通知”，而 CyclicBarrier 是“大家互相等，凑齐了再走”。

Java 中 ReentrantLock 的实现原理是什么？

ReentrantLock 的底层依赖 AQS (AbstractQueuedSynchronizer) 来实现线程的排队与阻塞控制。它本质上是一个同步器框架，通过一个 volatile 修饰的 state 变量来表示锁的状态。

- 1) 当线程尝试获取锁时，会通过 CAS 操作将 state 从 0 改为 1。如果修改成功，说明当前线程拿到了锁，并把 exclusiveOwnerThread 设置为自己，支持可重入，同一线程重复加锁时 state 累加。
- 2) 如果锁已被占用，当前线程就会被构造成一个 Node 节点，加入到 AQS 的等待队列中。这个队列是双向链表结构，线程进入后会被 park 阻塞，等待前驱节点释放锁后唤醒。
- 3) 释放锁时，线程会将 state 减 1，直到 state 降为 0 才真正释放。随后会唤醒后继节点，由 AQS 调度下一个线程去竞争锁。

公平锁和非公平锁的区别就在这里：公平锁每次获取都会检查队列是否有等待者；非公平锁则上来就抢，不管有没有人排队，可能压根不经过队列。

代码上看核心就是 lock() 和 unlock()：

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    // 临界区
} finally {
    lock.unlock();
}
```

注意必须配合 try-finally 使用，否则容易发生死锁。像 ConcurrentHashMap 或线程池内部任务调度这类高并发场景，都用到了类似的机制来替代 synchronized，控制更精细。

什么是 Java 的 CompletableFuture？

Java 里的 CompletableFuture 是为了把异步编程变得更简单而设计的。你以前用 Future 的时候，取结果得阻塞地调 get()，没法链式处理，特别不方便。CompletableFuture 在此基础上加了回调机制，可以指定任务完成后自动执行什么操作。

它实现了 Future 和 CompletionStage 两个接口，支持方法链式调用，比如 thenApply、thenAccept、thenRun 这些，能把多个异步任务串起来或者并行组合。不用自己去 join() 或者写一堆监听逻辑。

常见用法像这样：

```
CompletableFuture.supplyAsync(() -> {
    // 异步获取用户信息
    return userService.getUser(1);
})
```

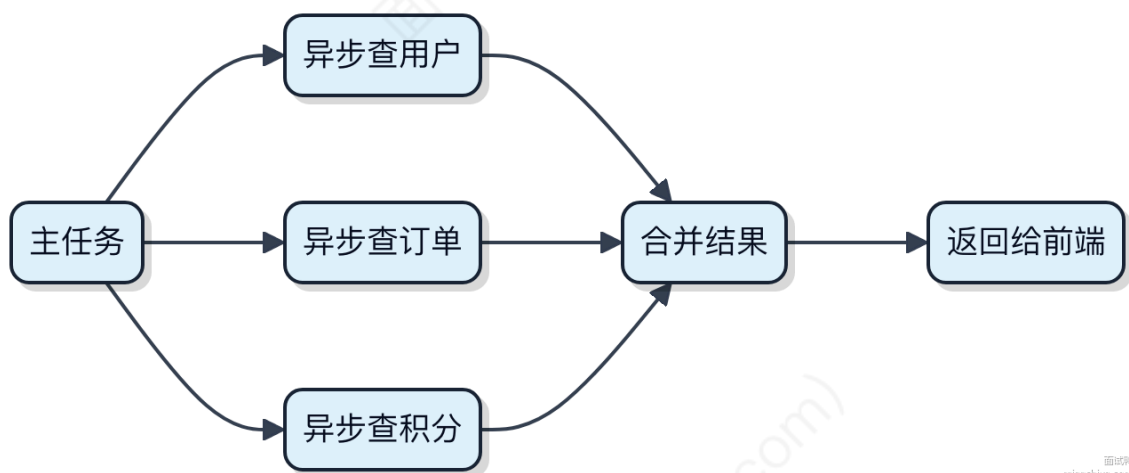
```
}).thenApply(user -> user.getName())
    .thenAccept(name -> System.out.println("Hello: " + name));
```

还可以处理异常，用 `exceptionally` 指定出错时的降级值：

```
}).exceptionally(e -> {
    log.error("查询失败", e);
    return "defaultUser";
});
```

如果要做聚合，比如等所有任务完成，可以用 `allOf`；只要有一个完成就返回，用 `anyOf`。像做批量数据拉取，配合线程池使用，能显著提升响应速度。

实际场景中，你在写一个接口要查用户、订单、积分三个服务，传统方式是顺序调用耗时 300ms+，用 `CompletableFuture` 并行发起请求，压根不经过主线程等待，最后合并结果，总耗时可能就 120ms。



面试网
mianshiya.com

什么是 Java 的 ForkJoinPool?

Java 的 `ForkJoinPool` 是为分治算法量身打造的线程池，特别适合能把大任务拆成小任务，最后再合并结果的场景。比如归并排序、遍历超大目录树，或者递归处理数据。

它最特别的地方是工作窃取机制。每个线程有自己的双端队列，任务都压到自己队列的头部。当某个线程干完活，不会闲着，而是从别人队列尾部“偷”任务来执行。这样能充分利用 CPU，避免线程饥饿。

普通线程池的任务都放在公共队列里，所有线程争抢一个队头，容易有竞争。`ForkJoinPool` 把调度的脏活累活分散到每个线程自己身上，扩展性更好。

用的时候得继承 `RecursiveTask` 或 `RecursiveAction`，重写 `compute()` 方法。任务大小有个阈值，别拆太细，否则调度开销可能超过收益。

```
class SumTask extends RecursiveTask<Long> {
    private final long[] arr;
    private final int start, end;
    private static final int THRESHOLD = 1000;
```

```
public Long compute() {
    if (end - start <= THRESHOLD) {
        return computeDirectly();
    }
    int mid = (start + end) >>> 1;
    SumTask left = new SumTask(arr, start, mid);
    SumTask right = new SumTask(arr, mid, end);
    left.fork();
    return right.compute() + left.join();
}
```

一般直接用 `ForkJoinPool.commonPool()`，这个公共实例默认线程数等于 CPU 核心数，大部分情况够用了。但 IO 密集型别瞎用，它设计给的是纯计算型任务。

本资源来自面试鸭：<https://www.mianshiya.com>

推荐更多免费学编程资源：

1. 编程导航学习网站：[学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库：[开始刷题，面试遇原题!](#)
3. 精选简历模板大全：[1 分钟搞定简历!](#)
4. AI 资源导航网站：[获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试：[随时随地提升面试能力](#)